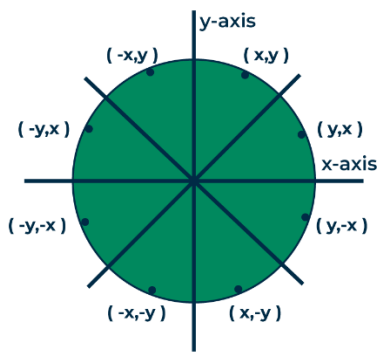


Unit 2. Line, Circle, and Polygon.

2.2 Circle generating algorithms:

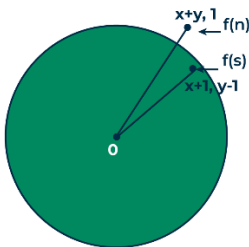
Bresenham's circle drawing algorithm

So, for whole 360 degree of circle we will divide it in 8-parts each octant of 45 degree. In order to do that we will use Bresenham's Circle Algorithm for calculation of the locations of the pixels in the first octant of 45 degrees. It assumes that the circle is centered on the origin. So for every pixel (x, y) it calculates, we draw a pixel in each of the 8 octants of the circle as shown below :



For a pixel (x, y) , all possible pixels in 8 octants

Now, we will see how to calculate the next pixel location from a previously known pixel location (x, y) . In Bresenham's algorithm at any point (x, y) we have two options either to choose the next pixel in the east i.e. $(x+1, y)$ or in the south east i.e. $(x+1, y-1)$.



And this can be decided by using the decision parameter d as:

- If $d > 0$, then $(x+1, y-1)$ is to be chosen as the next pixel as it will be closer to the arc.
- else $(x+1, y)$ is to be chosen as next pixel.

Now to draw the circle for a given radius ' r ' and centre (x_c, y_c) We will start from $(0, r)$ and move in first quadrant till $x=y$ (i.e. 45 degree). We should start from listed initial condition:

$$d = 3 - (2 * r)$$

$$x = 0$$

$$y = r$$

Now for each pixel, we will do the following operations:

1. Set initial values of (x_c, y_c) and (x, y) .
2. Set the decision parameter d to $d = 3 - (2 * r)$.
3. Call the `drawCircle(int xc, int yc, int x, int y)` function.
4. Repeat the following steps until $x \leq y$:
 - If $d < 0$, set $d = d + (4 * x) + 6$.
 - Else, set $d = d + 4 * (x - y) + 10$ and decrement y by 1.
 - Increment the value of x .
 - Call the `drawCircle(int xc, int yc, int x, int y)` function.

drawCircle() function:

// function to draw all other 7 pixels

// present at symmetric position

`drawCircle(int xc, int yc, int x, int y)`

```

{
    putpixel(xc+x, yc+y, RED);
    putpixel(xc-x, yc+y, RED);
    putpixel(xc+x, yc-y, RED);
    putpixel(xc-x, yc-y, RED);
    putpixel(xc+y, yc+x, RED);
    putpixel(xc-y, yc+x, RED);
    putpixel(xc+y, yc-x, RED);
    putpixel(xc-y, yc-x, RED);
}

```

Below is C implementation of above approach.

```

// C-program for circle drawing
// using Bresenham's Algorithm
// in computer-graphics

#include <stdio.h>
#include <dos.h>
#include <graphics.h>

// Function to put pixels
// at subsequence points

void drawCircle(int xc, int yc, int x, int y){
    putpixel(xc+x, yc+y, RED);

```

```
    putpixel(xc-x, yc+y, RED);  
  
    putpixel(xc+x, yc-y, RED);  
  
    putpixel(xc-x, yc-y, RED);  
  
    putpixel(xc+y, yc+x, RED);  
  
    putpixel(xc-y, yc+x, RED);  
  
    putpixel(xc+y, yc-x, RED);  
  
    putpixel(xc-y, yc-x, RED);  
}
```

```
// Function for circle-generation
```

```
// using Bresenham's algorithm
```

```
void circleBres(int xc, int yc, int r){
```

```
    int x = 0, y = r;
```

```
    int d = 3 - 2 * r;
```

```
    drawCircle(xc, yc, x, y);
```

```
    while (y >= x){
```

```
        // check for decision parameter
```

```
        // and correspondingly
```

```
        // update d, y
```

```
        if (d > 0) {
```

```
            y--;
```

```

        d = d + 4 * (x - y) + 10;
    }

    else

        d = d + 4 * x + 6;

    // Increment x after updating decision parameter

    x++;

    // Draw the circle using the new coordinates

    drawCircle(xc, yc, x, y);

    delay(50);
}
}

int main()
{
    int xc = 50, yc = 50, r = 30;

    int gd = DETECT, gm;

    initgraph(&gd, &gm, ""); // initialize graph

    circleBres(xc, yc, r); // function call

    return 0;
}

```

Output:



Advantages

- It is a simple algorithm.
- It can be implemented easily
- It is totally based on the equation of circle i.e. $x^2 + y^2 = r^2$

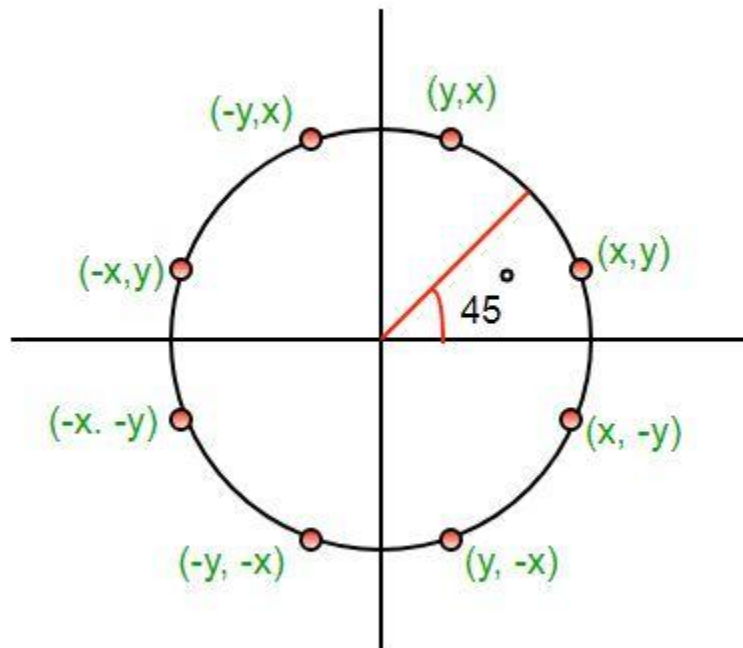
Disadvantages

- There is a problem of accuracy while generating points.
- This algorithm is not suitable for complex and high graphic images.

Mid-Point Circle Drawing Algorithm

The **mid-point** circle drawing algorithm is an algorithm used to determine the points needed for rasterizing a circle.

We use the **mid-point** algorithm to calculate all the perimeter points of the circle in the **first octant** and then print them along with their mirror points in the other octants. This will work because a circle is symmetric about its centre.



The algorithm is very similar to the [Mid-Point Line Generation Algorithm](#). Here, only the boundary condition is different.

For any given pixel (x, y) , the next pixel to be plotted is either $(x, y+1)$ or $(x-1, y+1)$. This can be decided by following the steps below.

1. Find the mid-point p of the two possible pixels i.e $(x-0.5, y+1)$
2. If p lies inside or on the circle perimeter, we plot the pixel $(x, y+1)$, otherwise if it's outside we plot the pixel $(x-1, y+1)$

Boundary Condition : Whether the mid-point lies inside or outside the circle can be decided by using the formula:-

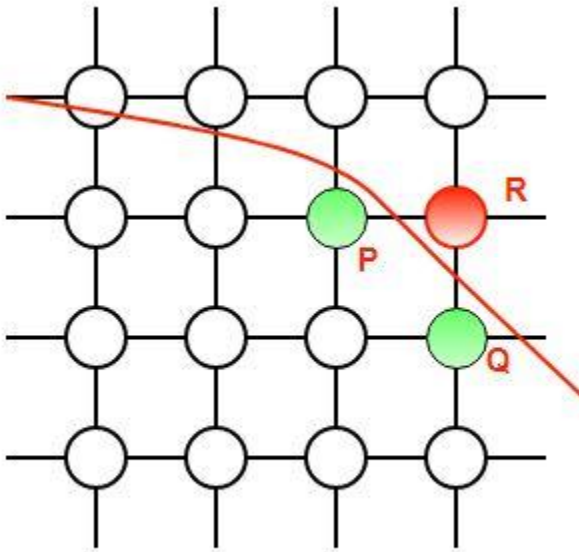
Given a circle centered at (0,0) and radius r and a point $p(x,y)$

$$F(p) = x^2 + y^2 - r^2$$

if $F(p) < 0$, the point is inside the circle

$F(p) = 0$, the point is on the perimeter

$F(p) > 0$, the point is outside the circle



In our program, we denote $F(p)$ with P . The value of P is calculated at the mid-point of the two contending pixels i.e. $(x-0.5, y+1)$. Each pixel is described with a subscript k .

$$P_k = (X_k - 0.5)^2 + (y_k + 1)^2 - r^2$$

Now,

$$x_{k+1} = x_k \text{ or } x_{k-1}, y_{k+1} = y_k + 1$$

$$? P_{k+1} = (x_{k+1} - 0.5)^2 + (y_{k+1} + 1)^2 - r^2$$

$$= (x_{k+1} - 0.5)^2 + [(y_k + 1) + 1]^2 - r^2$$

$$= (x_{k+1} - 0.5)^2 + (y_k + 1)^2 + 2(y_k + 1) + 1 - r^2$$

$$= (x_{k+1} - 0.5)^2 + [-(x_k - 0.5)^2 + (x_k - 0.5)^2] + (y_k + 1)^2 - r^2 + 2(y_k + 1) + 1$$

$$= P_k + (x_{k+1} - 0.5)^2 - (x_k - 0.5)^2 + 2(y_k + 1) + 1$$

$$= P_k + (x_{k+1}^2 - x_k^2) - (x_{k+1} - x_k) + 2(y_k + 1) + 1$$

$$= P_k + 2(y_k + 1) + 1, \text{ when } P_k \leq 0 \text{ i.e. the midpoint is inside the circle}$$

$$(x_{k+1} = x_k)$$

$P_k + 2(y_k + 1) - 2(x_k - 1) + 1$, when $P_k > 0$ i.e the mid point is outside the circle ($x_{k+1} = x_k - 1$)

The first point to be plotted is (r, 0) on the x-axis. The initial value of P is calculated as follows:-

$$\begin{aligned} P_1 &= (r - 0.5)^2 + (0+1)^2 - r^2 \\ &= 1.25 - r \\ &= 1 - r \text{ (When rounded off)} \end{aligned}$$

Examples:

Input : Centre -> (0, 0), Radius -> 3

Output : (3, 0) (3, 0) (0, 3) (0, 3)

(3, 1) (-3, 1) (3, -1) (-3, -1)

(1, 3) (-1, 3) (1, -3) (-1, -3)

(2, 2) (-2, 2) (2, -2) (-2, -2)

(x1,y1) is initially printed before the loop: (3,0) (3,0) (0,3) (0,3)

k	P_k	X_k	Y_k	P_{k+1}	X_{k+1}	Y_{k+1}	Output
1	-2	3	0	-1	3	1	(3,1) (-3,1) (3,-1) (-3,-1) (1,3) (-1,3) (1,-3) (-1,-3)
2	-1	3	1	2	2	2	(2,2) (-2,2) (2,-2) (-2,-2)
3	2	2	2				Break from loop

Input : Centre -> (4, 4), Radius -> 2

Output : (6, 4) (6, 4) (4, 6) (4, 6)

(6, 5) (2, 5) (6, 3) (2, 3)

(5, 6) (3, 6) (5, 2) (3, 2)

2.3 Polygons – Types of polygons, Inside Outside test, polygon filling: Seed fill algorithms: flood fill, boundry fill, scan-line algorithms.

A polygon is a closed [two-dimensional figure](#) composed of straight-line segments that meet at their endpoints. The line segments of polygons are called sides, and each endpoint is called a vertex. Polygons have at least three sides and three angles, and their sides must be straight, as seen below.

A polygon cannot have any curve sides. Open-sided figures and shapes with curves are examples of non-polygon figures, as seen below.

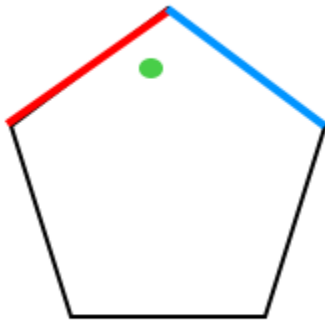
Polygon is an ordered list of vertices as shown in the following figure. For filling polygons with particular colors, you need to determine the pixels falling on the border of the polygon and those which fall inside the polygon. In this chapter, we will see how we can fill polygons using different techniques.



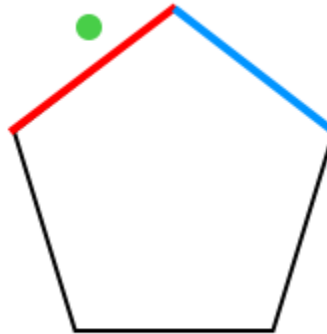
How to check if a given point lies inside or outside a polygon?

Given a **polygon** and a point '**p**', find if '**p**' lies inside the polygon or not. The points lying on the border are considered inside.

Examples:



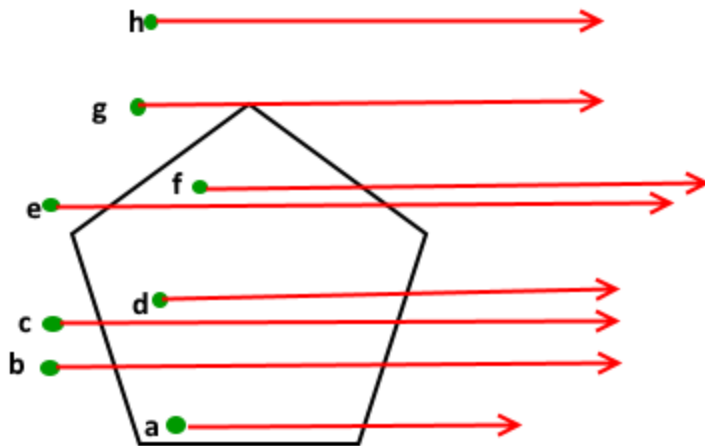
Inside example



outside example

Approach: The idea to solve this problem is based on [How to check if two given line segments intersect](#), and to be used as follows:

1. Draw a horizontal line to the right of each point and extend it to infinity
2. Count the number of times the line intersects with polygon edges.
3. A point is inside the polygon if either count of intersections is odd or point lies on an edge of polygon. If none of the conditions is true, then point lies outside.



How to handle point 'g' in the above figure?

Note that we should return true if the point lies on the line or the same as one of the vertices of the given polygon. To handle this, after checking if the line from 'p' to extreme intersects, we check whether 'p' is collinear with vertices of the current line of polygon. If it is collinear, then we check if the point 'p' lies on current side of polygon, if it lies, we return true, else false.

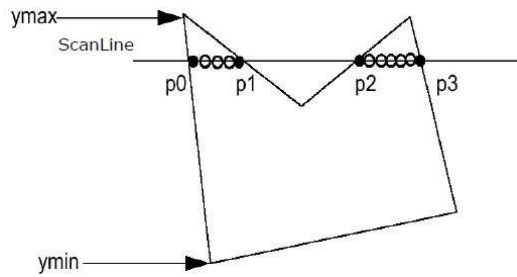
Seed fill algorithms

Seed fill algorithms are used to fill a connected region in an image, starting from a given seed point. These algorithms are commonly used in paint programs (like the "bucket fill" tool) and graphics rendering.

Scan Line Algorithm

This algorithm works by intersecting scanline with polygon edges and fills the polygon between pairs of intersections. The following steps depict how this algorithm works.

Step 1 – Find out the Ymin and Ymax from the given polygon.



Step 2 – ScanLine intersects with each edge of the polygon from Ymin to Ymax. Name each intersection point of the polygon. As per the figure shown above, they are named as p0, p1, p2, p3.

Step 3 – Sort the intersection point in the increasing order of X coordinate i.e. (p0, p1), (p1, p2), and (p2, p3).

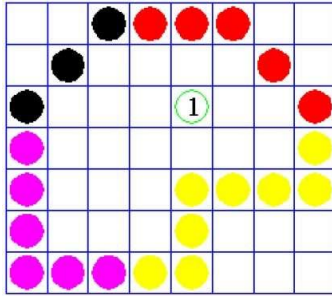
Step 4 – Fill all those pair of coordinates that are inside polygons and ignore the alternate pairs.

Flood Fill Algorithm

Sometimes we come across an object where we want to fill the area and its boundary with different colors. We can paint such objects with a specified interior color instead of searching for particular boundary color as in boundary filling algorithm.

Instead of relying on the boundary of the object, it relies on the fill color. In other words, it replaces the interior color of the object with the fill color. When no more pixels of the original interior color exist, the algorithm is completed.

Once again, this algorithm relies on the Four-connect or Eight-connect method of filling in the pixels. But instead of looking for the boundary color, it is looking for all adjacent pixels that are a part of the interior.



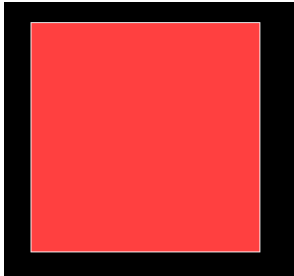
Given a rectangle, your task to fill this rectangle using flood fill algorithm.

Examples:

Input : rectangle(left = 50, top = 50, right= 100, bottom = 100)

flood(x = 55, y = 55, new_color = 12, old_color = 0)

Output :



Input : rectangle(left = 50, top = 50, right= 200, bottom = 400)

flood(x = 51, y = 51, new_color = 6, old_color = 0)

Output :



Flood fill algorithm fills new color until the old color match.

Flood fill algorithm:-

```
// A recursive function to replace previous
```

```
// color 'oldcolor' at '(x, y)' and all
```

```
// surrounding pixels of (x, y) with new
```

```
// color 'newcolor' and
```

```
floodfill(x, y, newcolor, oldcolor)
```

1) If x or y is outside the screen, then

return.

2) If color of getpixel(x, y) is same as

oldcolor, then

3) Recur for top, bottom, right and left.

```
floodFill(x+1, y, newcolor, oldcolor);
```

```
floodFill(x-1, y, newcolor, oldcolor);
```

```
floodFill(x, y+1, newcolor, oldcolor);
```

```
floodFill(x, y-1, newcolor, oldcolor);
```

Boundary Fill Algorithm

The boundary fill algorithm works as its name. This algorithm picks a point inside an object and starts to fill until it hits the boundary of the object. The color of the boundary and the color that we fill should be different for this algorithm to work.

Introduction : Boundary Fill Algorithm starts at a pixel inside the polygon to be filled and paints the interior proceeding outwards towards the boundary. This algorithm works **only if** the color with which the region has to be filled and the color of the boundary of the region are different. If the boundary is of one single color, this approach proceeds outwards pixel by pixel until it hits the boundary of the region.

Boundary Fill Algorithm is recursive in nature. It takes an interior point(x, y), a fill color, and a boundary color as the input. The algorithm starts by checking the color of (x, y). If it's color is not equal to the fill color and the boundary color, then it is painted with the fill color and the function is called for all the neighbours of (x, y). If a point is found to be of fill color or of boundary color, the function does not call its neighbours and returns. This process continues until all points up to the boundary color for the region have been tested.

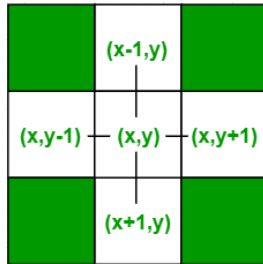
The boundary fill algorithm can be implemented by 4-connected pixels or 8-connected pixels.

4-connected pixels : After painting a pixel, the function is called for four neighbouring points. These are the pixel positions that are right, left, above, and below the current pixel. Areas filled by this method are called 4-connected. Below given is the algorithm :

Algorithm :

```
void boundaryFill4(int x, int y, int fill_color,int boundary_color)
```

```
{  
  
    if(getpixel(x, y) != boundary_color &&  
        getpixel(x, y) != fill_color)  
    {  
        putpixel(x, y, fill_color);  
  
        boundaryFill4(x + 1, y, fill_color, boundary_color);  
  
        boundaryFill4(x, y + 1, fill_color, boundary_color);  
  
        boundaryFill4(x - 1, y, fill_color, boundary_color);  
  
        boundaryFill4(x, y - 1, fill_color, boundary_color);  
    }  
}
```



Below is the implementation of above algorithm :

```
// C Implementation for Boundary Filling Algorithm

#include <graphics.h>

// Function for 4 connected Pixels

void boundaryFill4(int x, int y, int fill_color,int boundary_color)
{
    if(getpixel(x, y) != boundary_color &&
        getpixel(x, y) != fill_color)
    {
        putpixel(x, y, fill_color);

        boundaryFill4(x + 1, y, fill_color, boundary_color);

        boundaryFill4(x, y + 1, fill_color, boundary_color);

        boundaryFill4(x - 1, y, fill_color, boundary_color);

        boundaryFill4(x, y - 1, fill_color, boundary_color);
    }
}
```

```
    }  
}  
  
//driver code  
  
int main()  
{  
    // gm is Graphics mode which is  
    // a computer display mode that  
    // generates image using pixels.  
    // DETECT is a macro defined in  
    // "graphics.h" header file  
  
    int gd = DETECT, gm;  
  
    // initgraph initializes the  
    // graphics system by loading a  
    // graphics driver from disk  
    initgraph(&gd, &gm, "");  
  
    int x = 250, y = 200, radius = 50;  
  
    // circle function
```

```
circle(x, y, radius);

// Function calling
boundaryFill4(x, y, 6, 15);

delay(10000);

getch();

// closegraph function closes the
// graphics mode and deallocates
// all memory allocated by
// graphics system .
closegraph();

return 0;
}
```

Output :

Graphics Output Primitives

Plotting Point

Point is the fundamental element of the picture representation.

- It is nothing but the position in a plan defined as either pairs or triplets of number depending on whether the data are two or three dimensional.
- Thus, (x_1, y_1) or (x_1, y_1, z_1) would represent a point in either two three dimensional space.
 - Two points would represent a line or edge,
 - A collection of three or more points a polygon.
- The *representation of curved lines* is usually accomplished by approximating them by *short straight line segments*.

Graphics Output Primitives

If the two points used to specify a line are (x_1, y_1) and (x_2, y_2) , then an equation for the line is given as.

$$\frac{y - y_1}{x - x_1} = \frac{y_2 - y_1}{x_2 - x_1}$$

$$y = \frac{y_2 - y_1}{x_2 - x_1} (x - x_1) + y_1$$

$$Y = m x + b$$

Where $m = \frac{y_2 - y_1}{x_2 - x_1}$ And $b = y_1 - m x_1$

- The above equation is called *the **slop intercept form of the line***.
- The slope **m** is the change in height $(y_2 - y_1)$ divided by the change in the width $(x_2 - x_1)$ for two points on the line.
- The intercept **b** is the height at which the line crosses the y-axis.

Example :

Write an equation in slope-intercept form of the line that passes through the points (2,2) and (3,4).

➤ 'm' represents the slope

4 is y_2 and 2 is y_1

3 is x_2 and 2 is x_1

$$m = \frac{y_2 - y_1}{x_2 - x_1}$$

$$m = \frac{4 - 2}{3 - 2}$$

$$m =$$

$$m = 2$$

➤ Use the slope-intercept form of a line and the values we know to solve for 'b', which is the y-intercept

We can plug in 2 for m, and we take one of our ordered pairs and plug it in for x and y we solve b

$$Y = m x + b$$

$$Y = 2 x + b$$

$$4 = 2 (3) + b$$

$$4 = 6 + b$$

$$b = 4 -$$

$$b = -2$$

'b' is equal to -2, our y-intercept

Plug the values for slope and y-intercept into the slope-intercept form to find the equation of our line

$$Y = 2x - 2$$

4-1

Write the slope-intercept form of the equation of the line that passes through the two points.

1. $(2, 3), (6, 11)$

$$y = 2x - 1$$

2. $(1, -7), (3, -15)$

$$y = -4x - 3$$

Write an equation of the line in point-slope form that passes through the point and has the given slope. Then rewrite the equation in slope-intercept form.

3. $(-1, 1), m = \frac{2}{3}$

$$y = \frac{2}{3}x + \frac{5}{3}$$

4. $(6, -3), m = -\frac{1}{2}$

$$y = -\frac{1}{2}x$$

Write the equation in standard form of the line that passes through the two points.

5. $(5, 8), (3, 2)$

$$3x - y = 7$$

6. $(-4, -5), (-2, 5)$

$$5x - y = -15$$

- We can determine whether two lines are crossing or parallel.
- When two lines cross, they share some point in common. Of course, that point satisfies equations for the two lines. Let us see how to obtain this point.

Consider two lines having

slopes m_1 and m_2 and

y-intercepts b_1 and b_2 respectively.

Then we can write the equations for these two lines as,

Line 1 : $y = m_1 x + b_1$

Line 2: $y = m_2 x + b_2$

If point (x_p, y_p) is shared by both lines, then it should satisfy the last two equations

And $y_p = m_1 x_p + b_1$

$y_p = m_2 x_p + b_2$

Equating the above equations gives,

$$m_1 x_p + b_1 = m_2 x_p + b_2$$

$$x_p(m_1 - m_2) = b_2 - b_1$$

$$x_p = \frac{b_2 - b_1}{m_1 - m_2}$$

Substituting this into equation for line 1 or for line 2 gives,

$$y_p = m_1 \left[\frac{b_2 - b_1}{m_1 - m_2} \right] + b_1$$

$$y_p = \frac{m_1 (b_2 - b_1) + b_1 (m_1 - m_2)}{m_1 - m_2}$$

$$y_p = \frac{m_1 b_2 - m_1^2 b_1 + b_1 m_1 - b_1 m_2}{m_1 - m_2}$$

$$y_p = \frac{m_1 b_2 - b_1 m_2}{m_1 - m_2}$$

Therefore, the point $\left[\frac{b_2 - b_1}{m_1 - m_2}, \frac{m_1 b_2 - b_1 m_2}{m_1 - m_2} \right]$ is the point of intersection.

If the two lines are parallel, they have the same slope. In the expression for the point of intersection, $m_1 = m_2$. So the denominator becomes zero. So the expression results in a divide by zero. This means, there is no point of intersection for the parallel lines.

Line Drawing Algorithms

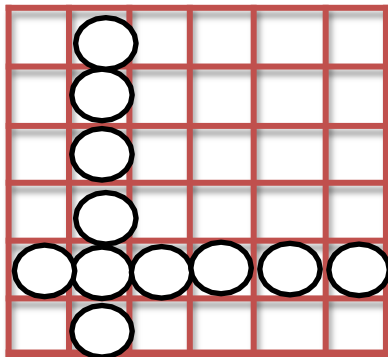
The process of ‘turning on’ the pixels for a line segment is called **vector generation** or **line generation**, and the algorithms for them are known as **vector generation algorithms** or **line drawing algorithms**.

Before discussing specific line drawing algorithms it is useful to note the general requirements for such algorithms. These requirements specify the desired characteristics of line.

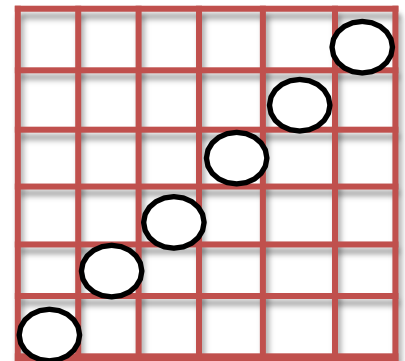
- The line should appear as a straight line and it should start and end accurately.
- The line should be displayed with constant brightness along its length independent of its length and orientation.
- The line should be drawn rapidly.

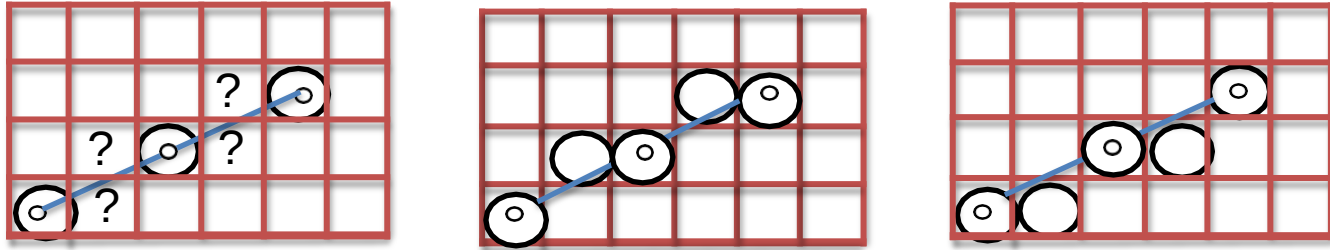
Let us see the different lines drawn in fig bellow,

Vertical and
Horizontal line



45° line





Line with other orientation

- As shown in the above figure, horizontal and vertical lines are straight and have same width.
- The 45° line is straight, but its width is not constant.
- The line with any other orientation is neither straight nor has same width. Such cases are due to the finite resolution of display and we have to accept approximate pixels in such situation.
- The brightness of the line is dependent on the orientation of the line. We can observe that the effective spacing between pixels for the 45° line is greater than for the vertical and horizontal lines. This will make the vertical and horizontal lines appear brighter than the 45° line.
- Complex calculations are required to provide equal brightness along lines of varying length and orientation.

Vector Generation/Digital Differential Analyzer (DDA) lineAlgorithm

The DDA algorithm generates lines from their differential equations.

- We calculate the **length of the line** in the **X** direction (number of pointes) by the equation:

$$\text{ABS}(x_2 - x_1)$$

- And calculate the **length of the line** in the **Y** direction (number of pointes) by the Equation:

$$\text{ABS}(y_2 - y_1)$$

Where ABS is a function takes the positive of the arguments.

The increment steps (Δx and Δy) are used to increment the x and y coordinates for the next pointes to be plotted.

$$\Delta x = (x_2 - x_1) / \text{length},$$

$$\Delta y = (y_2 - y_1) / \text{length}$$

Algorithm DDA

1. Read the line end points (x_1, y_1) and (x_2, y_2) such that they are not equal. [If equal then plot that point and exit]
2. $\Delta x = |x_2 - x_1|$ and $\Delta y = |y_2 - y_1|$
3. If $(\Delta x \geq \Delta y)$ then
 Length = Δx
Else
 Length = Δy
End if
4. $\Delta x = (x_2 - x_1) / \text{length}$
 $\Delta y = (y_2 - y_1) / \text{length}$

[This makes either Δx or Δy equal to 1 because length is either $x_2 - x_1$ or $y_2 - y_1$. Therefore, the incremental value for either x or y is one.]

5. $x = x_1 + 0.5 \text{ sign}(\Delta x)$
 $y = y_1 + 0.5 \text{ sign}(\Delta y)$

[Here, sign function makes the algorithm work in all quadrants. It returns -1, 0, 1 depending on whether its argument is <0 , $=0$, >0 , respectively.]

6. $i = 1$ [begins the loop, in this points are plotted]
 While ($i \leq \text{length}$)
 Begin
 Plot (integer (x), integer (y))
 $x = x + \Delta x$
 $y = y + \Delta y$
 $i = i + 1$
 End.

Stop

Note :

1- Sign function returns

- : -1 if its argument is < 0
- : 0 if its arguments is $= 0$
- : +1 if its arguments is > 0

Ex. $\text{Sign}(-10) = -1$, $\text{Sign}(5) = 1$

Using the Sign function makes the algorithm work in all quadrants.

Example :

consider the line from (0,0) to (6,6). Use the simple DDA algorithm to rasterize this line

Solution:

$$x1=0, y1=0, x2=6, y2=6$$

$$\text{Length} = y2 - y1 = x2 - x1 = 6$$

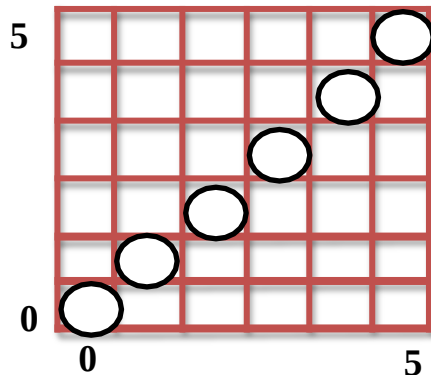
$$\Delta x = (x2 - x1) / \text{length} \\ = 6/6 = 1 \text{ and}$$

$$\Delta y = (y2 - y1) / \text{length} \\ = 6/6 = 1$$

Initial value for

$$x = 0 + 0.5 (1) = 0.5$$

$$Y = 0 + 0.5 (1) = 0.5$$



i	plot	x	y
		0.5	0.5
1	(0,0)		
		1.5	1.5
2	(1,1)		
		2.5	2.5
3	(2,2)		
		3.5	3.5
4	(3,3)		
		4.5	4.5
5	(4,4)		
		5.5	5.5
6	(5,5)		
		6.5	6.5

Example 2:

consider the line from (0,0) to (4,6). Use the simple DDA algorithm to rasterize this line

Solution:

$$x1=0, y1=0, x2=4, y2=6$$

$$\text{Length} = y2 - y1 = 6$$

$$\Delta x = (x2 - x1) / \text{length} \\ = 4/6 \quad \text{and}$$

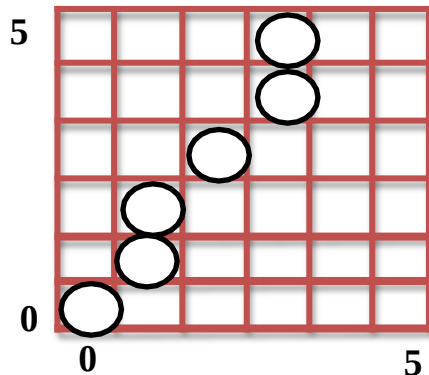
$$\Delta y = (y2 - y1) / \text{length} \\ = 6/6 = 1$$

Initial value for

$$x = 0 + 0.5 (4/6) = 0.5$$

$$Y = 0 + 0.5 (1) = 0.5$$

The result are plot ted as shown



i	plot	x	y
		0.5	0.5
1	(0,0)		
		1.167	1.5
2	(1,1)		
		1.83	2.5
3	(1,2)		
		2.5	3.5
4	(2,3)		
		3.167	4.5
5	(3,4)		
		3.833	5.5
6	(3,5)		
		4.5	6.5

Advantages of DDA Algorithm

1. It is the simplest algorithm and it does not require special skills for implementation.
2. It is a faster method for calculating pixel positions than the direct use of equation $y = mx + b$.

It eliminates the multiplication in the equation by making use of raster characteristics, so that appropriate increments are applied in the x or y direction to find the pixel positions along the line path.

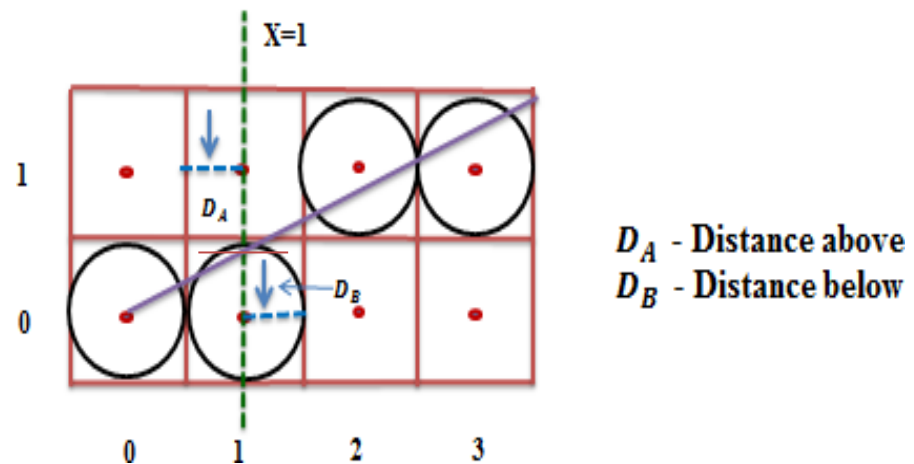
Disadvantages of DDA Algorithm

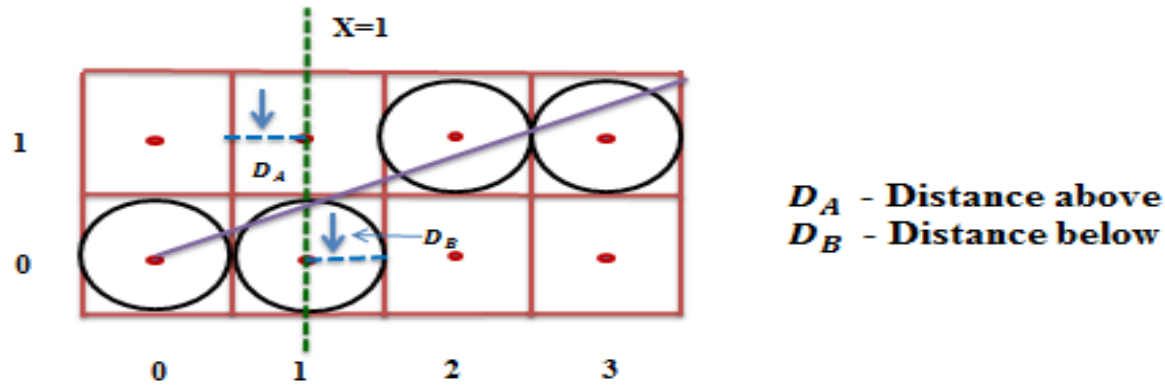
1. Floating point arithmetic in DDA algorithm is still time-consuming.
2. The algorithm is orientation dependent. So the end point accuracy is poor.

Bresenham's Line Algorithm

The *basic principle* of Bresenham's line algorithm is to **select the optimum raster locations** to **represent a straight line**.

- To accomplish this algorithm always **increments** either **x** or **y** by **one unit** **depending** on the **slope of line**.
- The increment in the other variable is determined by **examining the distance between** the **actual line location** and the **nearest pixel**.
- This **distance** is called **decision variable** or **the error**. This is illustrated in the Figure below.
- As shown above figure, the line **does not pass through all raster points (pixels)**. It **passes through raster point (0, 0)** and subsequently crosses three pixels.





It is seen that the intercept of line with the **line $x = 1$** is closer to the **line $y = 0$** , i.e. pixel (1, 0) than to the **line $y = 1$** i.e. pixel (1, 1).

- Hence, in this case, the **raster point** at (1, 0) **better represents the path of the line** than that at (1, 1).

The intercept of the line with the **line $x = 2$** is close to the **line $y = 1$** , i.e. pixel (2, 1) than to the **line $y = 0$** , i.e.: pixel (2, 0).

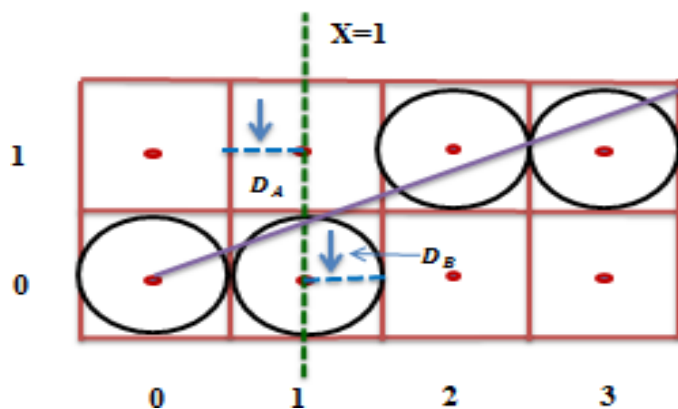
- Hence, the **raster point** at (2, 1) **better represents the path of the line**, as shown in the above figure.

In mathematical terms error or decision variable is defined as

$$e = D_B - D_A \quad \text{OR} \quad e = D_A - D_B$$

Let us define $e = D_B - D_A$. Now

- if ($e > 0$) then it implies that ($D_B > D_A$) i.e., the *pixel above the line* is *closer to the true line*.
- if ($e < 0$) then it implies that ($D_B < D_A$) i.e., the *pixel below the line* is *closer to the true line*.
- Thus by *checking* only the *sign of error* term it is possible to *determine the better pixel to represent the line path*.



D_A - Distance above
 D_B - Distance below

The error is initially set as

$$e = 2 \Delta y - \Delta x$$

Where

$$\Delta y = y_2 - y_1, \text{ and}$$

$$\Delta x = x_2 - x_1$$

Then according to value of e following actions are taken.

While ($e \geq 0$)

{

$$y = y + 1$$

$$e = e - 2 * \Delta x$$

}

$$x = x + 1$$

$$e = e + 2 * \Delta y$$

- When ($e \geq 0$), error is initialized with $e = e - 2 * \Delta x$. This is continued until **error is negative**. In each iteration y is *increment by 1*.
- When ($e < 0$), error is initialized to $e = e + 2 * \Delta y$. In both the cases x is *incremented by 1*.

Bresenham's Line Algorithm

1. Read the line end points (x_1, y_1) and (x_2, y_2) such that they are not equal.
[If equal, then plot that point and exit]
2. $\Delta x = |x_2 - x_1|$ and $\Delta y = |y_2 - y_1|$
3. $x = x_1$ [Initialize starting point]
 $y = y_1$
4. $e = 2 * \Delta y - \Delta x$ [Initialize value of decision variable or error to compensate for non zero intercepts]
5. $i = 1$ [Initialize counter]
6. Plot (x, y)
7. While $(e \geq 0)$
{
 $y = y + 1$
 $e = e - 2 * \Delta x$
}
 $x = x + 1$
 $e = e + 2 * \Delta y$
8. $i = i + 1$
9. If $(i \leq \Delta x)$ then go to step 6
10. Stop

Example:- Consider the line from (5, 5) to (13, 9), use the Bresenham's algorithm to rasterize the line

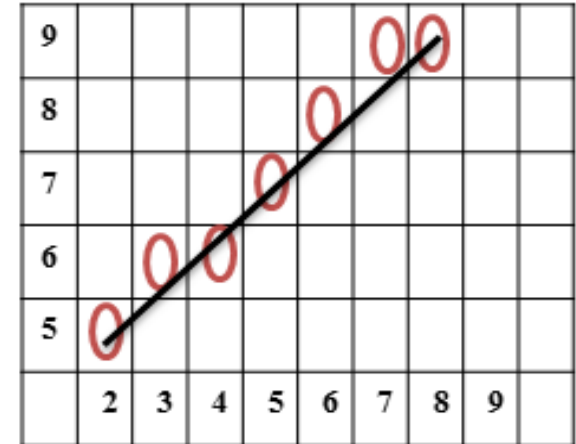
1. $\Delta x = |x_2 - x_1|$
2. $\Delta y = |y_2 - y_1|$
3. $x = x_1$
 $y = y_1$
4. $e = 2 * \Delta y - \Delta x$
5. $i = 1$
6. Plot (x, y)
7. While ($e \geq 0$)
 - {
 - $y = y + 1$
 - $e = e - 2 * \Delta x$
 - }
 - $x = x + 1$
 - $e = e + 2 * \Delta y$
8. $i = i + 1$
9. If ($i \leq \Delta x$) then
go to step 6
10. Stop

$\Delta x = |9 - 2| = 7$
 $\Delta y = |10 - 5| = 5$
 $x = 2$
 $y = 5$
 $e = 2 * 5 - 7 = 10 - 7 = 3$
 $i = 1$

6: plot (2, 5)
 while ($e \geq 0$)
 {
 $y = 5 + 1 = 6$
 $e = 3 - 2 * 7 = -11$
 }
 $x = 2 + 1 = 3$
 $e = -11 + 2 * 5 = -1$
 $i = i + 1 = 2$
 if ($2 \leq 7$) then
 goto step 6

6: plot (3, 6)
 while ($e \geq 0$)
 {
 $y = 6$
 $e = -1$
 }
 $x = 3 + 1 = 4$
 $e = -1 + 2 * 5 = 9$
 $i = 2 + 1 = 3$
 if ($3 \leq 7$) then
 goto step 6

6: plot (4, 6)
 $y = 6 + 1 = 7$
 $e = 9 - 2 * 7 = -5$
 $x = 4 + 1 = 5$
 $e = -5 + 2 * 5 = 5$
 $i = 3 + 1 = 4$
 if ($4 \leq 7$)
 goto step 6



Example:- Consider the line from (5, 5) to (13, 9), use the Bresenham's algorithm to rasterize the line.

Solution: Evaluating steps 1 through 4 in the Bresenham's algorithm we have,

$$\Delta x = |13-5|=8$$

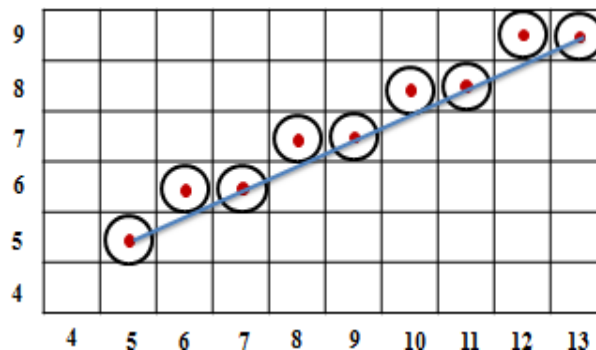
$$\Delta y = |9-5|=4$$

$$x=5, y=5.$$

$$e = 2 * \Delta y - \Delta x = 2 * 4 - 8 = 0$$

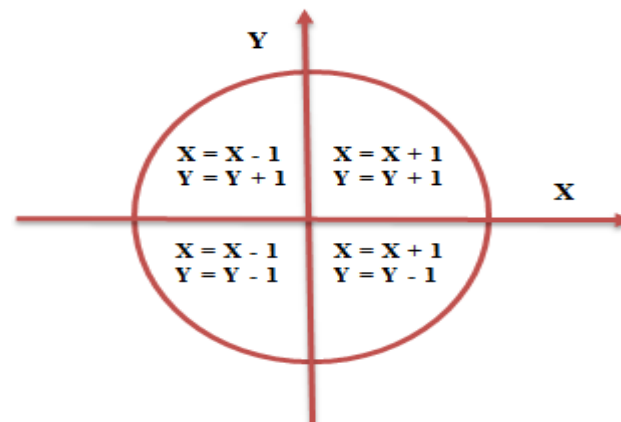
Tabulating the results of each iteration in the step 5 through 10

The results are plotted as shown in figure below



i	plot	x	y	e
		5	5	0
1	(5,5)	6	6	-16
				-8
2	(6,6)	7	6	0
3	(7,6)	8	7	-16
				-8
4	(8,7)	9	7	0
5	(9,7)	10	8	-16
				-8
6	(10,8)	11	8	0
7	(11,8)	12	9	-16
				-8
8	(12,9)	13	9	0
9	(13,9)	14	10	-16
				-8

- The Bresenham's algorithm *only works* for the *first octant*.
- The generalized Bresenham's algorithm *requires modification* for lines lying in the *other octants*.
- Such algorithm can be *easily developed* by *considering* the *quadrant* in which the *line lies* and *its slope*.
- When the *absolute magnitude* of the *slope of the line* is *greater than 1*, *y* is *incremented by one*.
- *Bresenham's decision variable* or *error* is used to *determine* when to *increment x*.
- The *x* and *y incremental values* depend on the *quadrant* in which the *line exists*. This is illustrated in below figure.



Conditions for generalized Bresenham's algorithm

Generalized Bresenham's Algorithm

1. Read the line end points (x_1, y_1) and (x_2, y_2) such that they are not equal.

[If equal then plot that point and exit]

2. $\Delta x = |x_2 - x_1|$ and $\Delta y = |y_2 - y_1|$

3. $x = x_1$ [Initialize starting point]
 $y = y_1$

4. $S_1 = \text{sign}(x_2 - x_1)$

$S_2 = \text{sign}(y_2 - y_1)$

5. If $\Delta y > \Delta x$ then

begin

$T = \Delta x : \Delta x = \Delta y : \Delta y = T : \text{Interchange} = 1$

end

else

Interchange=0

End If

6. $e = 2 * \Delta y - \Delta x$ [Initialize value of decision variable or error to compensate for non zero intercepts]

7. $i = 1$ [initialize counter]

8. Plot (x, y)

9. While $(e \geq 0)$

begin

if (Interchange=1) then $x = x + s_1$

else $y = y + s_2$

end If

$e = e - 2 * \Delta x$

end while

10. If (Interchange= 1), then $y = y + s_2$

else $x = x + s_1$

end If

$e = e + 2 * \Delta y$

11. $i = i + 1$

12. If ($i \leq \Delta x$) then go to step 8

13. Stop

Example : Draw the line from (0,0) to (-8,-4) using General Bresenham algorithm

$$\Delta x = | -8 - 0 | = 8$$

$$\Delta y = | -4 - 0 | = 4$$

$$x = x_1 \quad x_1 = 0$$

$$y = y_1 \quad y_1 = 0$$

$$S_1 = \text{sign}(x_2 - x_1) \\ = \text{sign}(-8 - 0) = \text{sign}(-8)$$

$$S_2 = \text{sign}(y_2 - y_1) \\ = \text{sign}(-4 - 0) = \text{sign}(-4)$$

$$x = 0 \quad y = 0$$

$$\Delta x = 8 \quad \Delta y = 4$$

$$S_1 = -1 \quad S_2 = -1$$

Because $\Delta x > \Delta y$ then

Interchange=0 :

$$e = 2 * \Delta y - \Delta x$$

$$e = 2 * 4 - 8$$

$$e = 0$$

i	plot	e	x	y
		0	0	0
1	(0,0)			
		-16	0	-1
		-8	-1	-1
2	(-1,-1)			
		0	-2	-1
3	(-2,-1)			
		-16	-2	-2
		-8	-3	-2
4	(-3,-2)			
		0	-4	-2
5	(-4,-2)			
		-16	-4	-3
		-8	-5	
6	(-5,-3)			
		0	-6	-3
7	(-6,-3)			
		-16	-6	-4
		-8	-7	-4
8	(-7,-4)			
		0	-8	-4

Unit 2
Line, Circle, and
Polygon.

2.1 Image Representation

Point is the fundamental element of the picture representation. It is nothing but the position in a plane defined as either pairs or triplets of numbers depending on whether the data are two or three dimensional. Thus, (x_1, y_1) or (x_1, y_1, z_1) would represent a point in either two or three dimensional space. Two points would represent a line or edge, and a collection of three or more points a polygon. The representation of curved lines is usually accomplished by approximating them by short straight line segments.

If the two points used to specify a line are (x_1, y_1) and (x_2, y_2) , then an equation for the line is given as

$$\frac{y-y_1}{x-x_1} = \frac{y_2-y_1}{x_2-x_1} \quad \dots (2.1)$$

$$\therefore y = \frac{y_2-y_1}{x_2-x_1} (x - x_1) + y_1 \quad \dots(2.2)$$

$$\text{or} \quad y = mx + b \quad \dots(2.3)$$

where

$$m = \frac{y_2-y_1}{x_2-x_1} \quad \dots(2.4)$$

$$\text{and} \quad b = y_1 - mx_1 \quad \dots(2.5)$$

The above equation is called the slope intercept form of the line. The slope m is the change in height $(y_2 - y_1)$ divided by the change in width $(x_2 - x_1)$ for two points on the line. The intercept b is the height at which the line crosses the y -axis.

Another form of the line equation, called the general form, may be found by multiplying out the factors and rearranging the equation (2.1).

$$(y_2 - y_1) X - (x_2 - x_1) y + x_2 y_1 - x_1 y_2 = 0 \quad \dots(2.6)$$

or

$$rx + sy + t = 0 \quad \dots (2.7)$$

2.3.1 Digital Differential Analyzer

We know that the slope of a straight line is given as

$$m = \frac{\Delta y}{\Delta x} = \frac{y_2 - y_1}{x_2 - x_1} \quad \dots (2.1)$$

The above differential equation can be used to obtain a rasterized straight line. For any given x interval Δx along a line, we can compute the corresponding y interval Δy from equation 2.1 as

$$\Delta y = \frac{y_2 - y_1}{x_2 - x_1} \Delta x \quad \dots (2.2)$$

Similarly, we can obtain the x interval Δx corresponding to a specified Δy as

$$\Delta x = \frac{x_2 - x_1}{y_2 - y_1} \Delta y \quad \dots (2.3)$$

Once the intervals are known the values for next x and next y on the straight line can be obtained as follows

$$\begin{aligned} x_{i+1} &= x_i + \Delta x \\ &= x_i + \frac{x_2 - x_1}{y_2 - y_1} \Delta y \end{aligned} \quad \dots (2.4)$$

$$\begin{aligned} \text{and } y_{i+1} &= y_i + \Delta y \\ &= y_i + \frac{y_2 - y_1}{x_2 - x_1} \Delta x \end{aligned} \quad \dots (2.5)$$

The equations 2.4 and 2.5 represent a recursion relation for successive values of x and y along the required line. Such a way of rasterizing a line is called a **digital differential analyzer (DDA)**. For simple DDA either Δx or Δy , whichever is larger, is chosen as one raster unit, i.e.

$$\text{if } |\Delta x| \geq |\Delta y| \text{ then}$$

differential analyzer (DDA). For simple DDA either Δx or Δy , whichever is larger, is chosen as one raster unit, i.e.

$$\text{if } |\Delta x| \geq |\Delta y| \text{ then}$$

$$\Delta x = 1$$

else

$$\Delta y = 1$$

With this simplification, if $\Delta x = 1$ then

$$\text{we have } y_{i+1} = y_i + \frac{y_2 - y_1}{x_2 - x_1} \text{ and}$$

$$x_{i+1} = x_i + 1$$

If $\Delta y = 1$ then

we have

$$y_{i+1} = y_i + 1 \text{ and}$$

$$x_{i+1} = x_i + \frac{x_2 - x_1}{y_2 - y_1}$$

DDA line Drawing Algorithm

1. Read the line end points (x_1, y_1) and (x_2, y_2) such that they are not equal.
[if equal then plot that point and exit]
2. $\Delta x = |x_2 - x_1|$ and $\Delta y = |y_2 - y_1|$
3. if $(\Delta x \geq \Delta y)$ then
 $\text{length} = \Delta x$
else
 $\text{length} = \Delta y$
end if
4. $\Delta x = (x_2 - x_1) / \text{length}$
 $\Delta y = (y_2 - y_1) / \text{length}$
[This makes either Δx or Δy equal to 1 because length is either $|x_2 - x_1|$ or $|y_2 - y_1|$. Therefore, the incremental value for either x or y is one.]
5. $x = x_1 + 0.5 * \text{Sign}(\Delta x)$
 $y = y_1 + 0.5 * \text{Sign}(\Delta y)$
[Here, Sign function makes the algorithm work in all quadrant. It returns -1, 0, 1 depending on whether its argument is < 0 , $= 0$, > 0 respectively. The factor 0.5 makes it possible to round the values in the integer function rather than truncating them.]
6. $i = 1$
 [Begins the loop, in this loop points are plotted]
 While ($i \leq \text{length}$)
 {
 Plot (Integer (x), Integer (y))
 $x = x + \Delta x$
 $y = y + \Delta y$
 $i = i + 1$
 }
7. Stop

Let us see few examples to illustrate this algorithm.

➡ **Example 2.1 :** Consider the line from (0, 0) to (4, 6). Use the simple DDA algorithm to rasterize this line.

Solution : Evaluating steps 1 to 5 in the DDA algorithm we have,

$$\begin{aligned}
 x_1 &= 0 & y_1 &= 0 & x_2 &= 4 & y_2 &= 6 \\
 \therefore \text{Length} &= |y_2 - y_1| = 6 \\
 \therefore \Delta x &= |x_2 - x_1| / \text{length} \\
 &= \frac{4}{6} \\
 \text{and } \Delta y &= |y_2 - y_1| / \text{length} \\
 &= 6 / 6 = 1
 \end{aligned}$$

Initial value for

$$x = 0 + 0.5 * \text{Sign} \left(\frac{4}{6} \right) = 0.5$$

$$y = 0 + 0.5 * \text{Sign} (1) = 0.5$$

Tabulating the results of each iteration in the step 6 we get,

i	Plot	x	y
1	(0, 0)	0.5	0.5
2	(1, 1)	1.167	1.5

3	(1, 2)	1.833	2.5
4	(2, 3)	2.5	3.5
5	(3, 4)	3.167	4.5
6	(3, 5)	3.833	5.5
		4.5	6.5

The results are plotted as shown in the Fig. 2.5. It shows that the rasterized line lies to both sides of the actual line, i.e. the algorithm is **orientation dependent**.

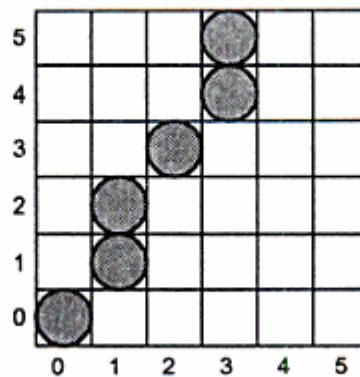


Fig. 2.5 Result for a simple DDA

➡ **Example 2.2 :** Consider the line from $(0, 0)$ to $(-6, -6)$. Use the simple DDA algorithm to rasterize this line.

Advantages of DDA Algorithm

1. It is the simplest algorithm and it does not require special skills for implementation.
2. It is a faster method for calculating pixel positions than the direct use of equation $y = mx + b$. It eliminates the multiplication in the equation by making use of raster characteristics, so that appropriate increments are applied in the x or y direction to find the pixel positions along the line path.

Disadvantages of DDA Algorithm

1. Floating point arithmetic in DDA algorithm is still time-consuming.
2. The algorithm is orientation dependent. Hence end point accuracy is poor.

2.2.2 Bresenham's Line Algorithm

Bresenham's line algorithm uses only integer addition and subtraction and multiplication by 2, and we know that the computer can perform the operations of integer addition and subtraction very rapidly. The computer is also time-efficient when performing integer multiplication by powers of 2. Therefore, it is an efficient method for scan-converting straight lines.

The basic principle of Bresenham's line algorithm is to select the optimum raster locations to represent a straight line. To accomplish this the algorithm always increments either x or y by one unit depending on the slope of line. The increment in the other variable is determined by examining the distance between the actual line location and the nearest pixel. This distance is called decision variable or the error. This is illustrated in the Fig. 2.7.

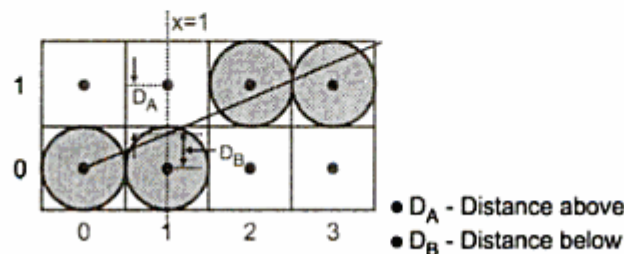


Fig. 2.7

As shown in the Fig. 2.7, the line does not pass through all raster points (pixels). It passes through raster point (0, 0) and subsequently crosses three pixels. It is seen that the intercept of line with the line $x = 1$ is closer to the line $y = 0$, i.e. pixel (1, 0) than to the line $y = 1$ i.e. pixel (1, 1). Hence, in this case, the raster point at (1, 0) better represents the path of the line than that at (1, 1). The intercept of the line with the line $x = 2$ is close to the line $y = 1$, i.e. pixel (2, 1) than to the line $y = 0$, i.e. pixel (2, 0). Hence, the raster point at (2, 1) better represents the path of the line, as shown in the Fig. 2.7.

In mathematical terms error or decision variable is defined as,

$$e = D_B - D_A \quad \text{or} \quad D_A - D_B$$

Let us define $e = D_B - D_A$. Now if $e > 0$, then it implies that $D_B > D_A$, i.e., the pixel above the line is closer to the true line. If $D_B < D_A$ (i.e. $e < 0$) then we can say that the pixel below the line is closer to the true line. Thus by checking only the sign of error term it is possible to determine the better pixel to represent the line path.

The error term is initially set as

$$e = 2 \Delta y - \Delta x$$

where $\Delta y = y_2 - y_1$, and $\Delta x = x_2 - x_1$

Then according to value of e following actions are taken.

```
while ( e ≥ 0)
{
    y = y + 1
    e = e - 2 * Δx
}
x = x + 1
e = e + 2 * Δy
```

When $e \geq 0$, error is initialized with $e = e - 2 \Delta x$. This is continued till error is negative. In each iteration y is incremented by 1. When $e < 0$, error is initialized to $e = e + 2 \Delta y$. In both the cases x is incremented by 1. Let us see the Bresenham's line drawing algorithm.

Bresenham's Line Algorithm

1. Read the line end points (x_1, y_1) and (x_2, y_2) such that they are not equal.
[if equal then plot that point and exit]
2. $\Delta x = |x_2 - x_1|$ and $\Delta y = |y_2 - y_1|$
3. [Initialize starting point]
 $x = x_1$
 $y = y_1$
4. $e = 2 * \Delta y - \Delta x$
[Initialize value of decision variable or error to compensate for nonzero intercepts]
5. $i = 1$ [Initialize counter]
6. Plot (x, y)
7. while ($e \geq 0$)
{
 $y = y + 1$
 $e = e - 2 * \Delta x$
}
 $x = x + 1$
 $e = e + 2 * \Delta y$
8. $i = i + 1$
9. if ($i \leq \Delta x$) then go to step 6.
10. Stop

►►► **Example 2.3 :** Consider the line from (5, 5) to (13, 9). Use the Bresenham's algorithm to rasterize the line.

Solution : Evaluating steps 1 through 4 in the Bresenham's algorithm we have,

$$\Delta x = |13 - 5| = 8$$

$$\Delta y = |9 - 5| = 4$$

$$x = 5$$

$$y = 5$$

$$\begin{aligned} e &= 2 * \Delta y - \Delta x = 2 * 4 - 8 \\ &= 0 \end{aligned}$$

Tabulating the results of each iteration in the step 5 through 10.

i	Plot	x	y	e
		5	5	0
1	(5, 5)	6	6	-8
2	(6, 6)	7	6	0
3	(7, 6)	8	7	-8
4	(8, 7)	9	7	0
5	(9, 7)	10	8	-8
6	(10, 8)	11	8	0
7	(11, 8)	12	9	-8
8	(12, 9)	13	9	0
9	(13, 9)	14	10	-8

Table 2.3

The results are plotted as shown in Fig. 2.8.

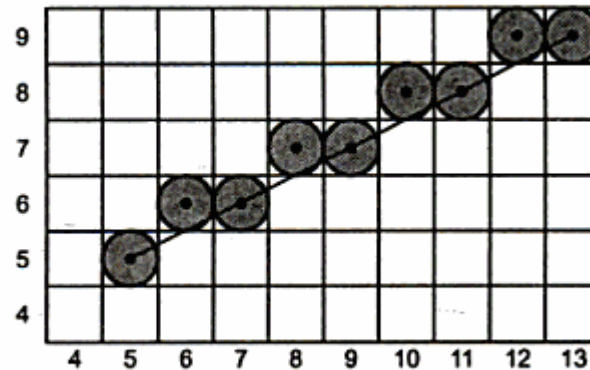


Fig. 2.8

2.7 Basic Concepts in Circle Drawing

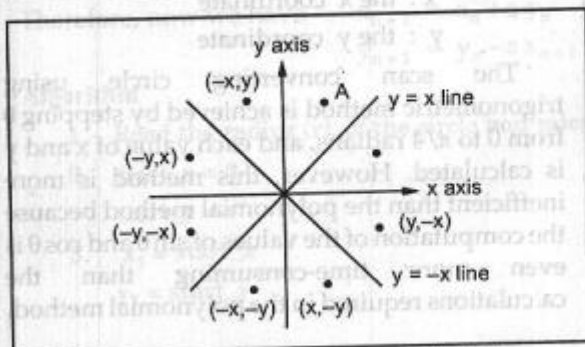


Fig. 2.13 Eight-way symmetry of a circle

A circle is a symmetrical figure. It has eight-way symmetry as shown in the Fig. 2.13. Thus, any circle generating algorithm can take advantage of the circle symmetry to plot eight points by calculating the coordinates of any one point. For example, if point A in the Fig. 2.13 is calculated with a circle algorithm, seven more points could be found just by reflection.

2.8 Representation of a Circle

There are two standard methods of mathematically representing a circle centered at the origin.

2.8.1 Polynomial Method

In this method circle is represented by a polynomial equation.

$$x^2 + y^2 = r^2$$

where x : the x coordinate

y : the y coordinate

r : radius of the circle

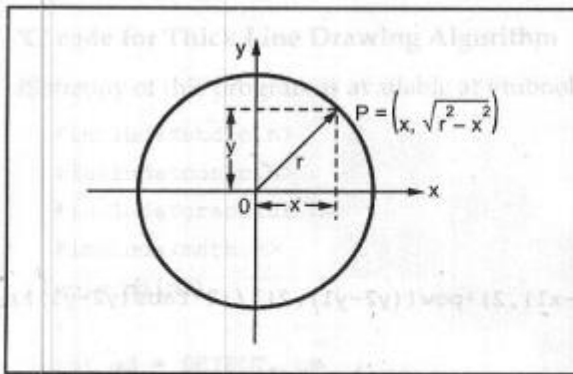


Fig. 2.14 Scan converting circle using polynomial method

Here, polynomial equation can be used to find y coordinate for the known x coordinate. Therefore, the scan converting circle using polynomial method is achieved by stepping x from 0 to $r\sqrt{2}$, and each y coordinate is found by evaluating $\sqrt{r^2 - x^2}$ for each step of x. This generates the 1/8 portion (90° to 45°) of the circle. Remaining part of the circle can be generated just by reflection.

this method for each point both x and r must be squared and x^2 must be subtracted from r^2 ; then the square root of the result must be found out.

The polynomial method of circle generation is an inefficient method. In

① Polynomial
 $x^2 + y^2 = r^2$
 $x^2 = r^2 - y^2$
 $x = \sqrt{r^2 - y^2}$
 for 45° $x = 0, y = r$
 for next pt from 0°
 ② $x = \sqrt{r^2 - r^2}$
 $= \sqrt{2r^2}$
 $x = 2\sqrt{2}$
 ③ $y = \sqrt{r^2 - x^2}$
 put values of x from each pt on ②

According to Pythagoras theorem

for P, $x = x$ then
 $r = \sqrt{r^2 - x^2}$

2.8.2. Trigonometric Method

In this method, the circle is represented by use of trigonometric functions

$$x = r \cos \theta \quad \text{and} \quad y = r \sin \theta$$

where θ : current angle

r : radius of the circle

x : the x coordinate

y : the y coordinate

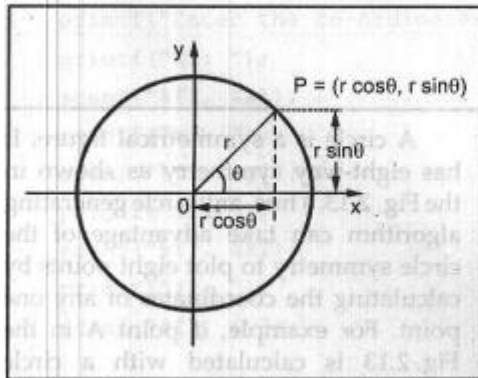


Fig. 2.15 Scan converting circle using trigonometric method

The scan converting circle using trigonometric method is achieved by stepping θ from 0 to $\pi/4$ radians, and each value of x and y is calculated. However, this method is more inefficient than the polynomial method because the computation of the values of $\sin \theta$ and $\cos \theta$ is even more time-consuming than the calculations required in the polynomial method.

2.9 Circle Drawing Algorithms

In this section we are going to discuss three efficient circle drawing algorithms :

- DDA algorithm and
- Bresenham's algorithm
- Midpoint algorithm

2.9.1 DDA Circle Drawing Algorithm

We know that, the equation of circle, with origin as the center of the circle is given as

$$x^2 + y^2 = r^2$$

The digital differential analyser algorithm can be used to draw the circle by defining circle as a differential equation. It is as given below

$$2x dx + 2y dy = 0 \quad \text{where } r \text{ is constant}$$

$$\therefore x dx + y dy = 0$$

$$\therefore y dy = -x dx$$

$$\therefore \frac{dy}{dx} = \frac{-x}{y}$$

From above equation, we can construct the circle by using incremental x value, $\Delta x = \epsilon y$ and incremental y value, $\Delta y = -\epsilon x$, where ϵ is calculated from the radius of the circle as given below

$$2^{n-1} \leq r < 2^n \quad r : \text{radius of the circle}$$

$$\epsilon = 2^{-n}$$

For example, if $r = 50$ then $n = 6$ so that $32 \leq 50 < 64$

$$\therefore \epsilon = 2^{-6}$$

$$= 0.0156$$

Applying these incremental steps we have,

$$x_{n+1} = x_n + \epsilon y_n$$

$$y_{n+1} = y_n - \epsilon x_n$$

Applying these incremental steps we have,

$$x_{n+1} = x_n + \epsilon y_n$$

$$y_{n+1} = y_n - \epsilon x_n$$

The points plotted using above equations give the spiral instead of the circle. To get the circle we have to make one correction in the equation; we have to replace x_n by x_{n+1} in the equation of y_{n+1} .

Therefore, now we have

$$x_{n+1} = x_n + \epsilon y_n$$

$$y_{n+1} = y_n - \epsilon x_{n+1}$$

$$\begin{aligned} 2^0 &= 1 \\ 2^1 &= 2 \\ 2^2 &= 4 \\ 2^3 &= 8 \\ 2^4 &= 16 \\ 2^5 &= 32 \\ 2^6 &= 64 \\ 2^7 &= 128 \\ 2^8 &= 256 \end{aligned}$$

For $r = 50$, $2 = ?$

$$2^{n-1} \leq r < 2^n$$

$$32 \leq 50 < 64$$

$$\therefore n = 6$$

$$\epsilon = 2^{-6}$$

Algorithm

1. Read the radius (r), of the circle and calculate value of ϵ
2. $\text{start_x} = 0$
 $\text{start_y} = r$
3. $x_1 = \text{start_x}$
 $y_1 = \text{start_y}$
4. do
 { $x_2 = x_1 + \epsilon y_1$
 $y_2 = y_1 - \epsilon x_2$
 [x_2 represents x_{n+1} and x_1 represents x_n]
 Plot (int (x_2), int (y_2))
 $x_1 = x_2$;
 $y_1 = y_2$;
 [Reinitialize the current point]
 } while ($y_1 - \text{start_y} < \epsilon$ or ($\text{start_x} - x_1 > \epsilon$)
 [check if the current point is the starting point or not. If current point is not starting point repeat step 4 ; otherwise stop]
5. Stop.

2.6.2 Bresenham's Circle Drawing Algorithm

The Bresenham's circle drawing algorithm considers the eight-way symmetry of the circle to generate it. It plots $1/8^{\text{th}}$ part of the circle, i.e. from 90° to 45° , as shown in the Fig. 2.24. As circle is drawn from 90° to 45° , the x moves in positive direction and y moves in the negative direction.

To achieve best approximation to the true circle we have to select those pixels in the raster that fall the least distance from the true circle. Refer Fig. 2.25. Let us observe the 90° to 45° portion of the circle. It can be noticed that if points are generated from 90° to 45° , each new point closest to the true circle can be found by applying either of the two options :

- Increment in positive x direction by one unit or
- Increment in positive x direction and negative y direction both by one unit.

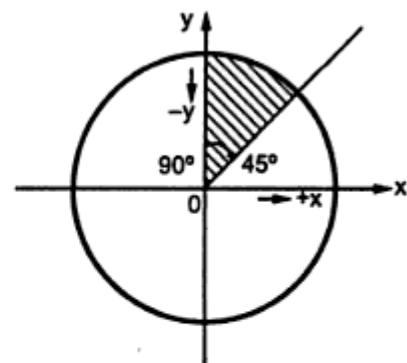


Fig. 2.24 $1/8$ part of circle

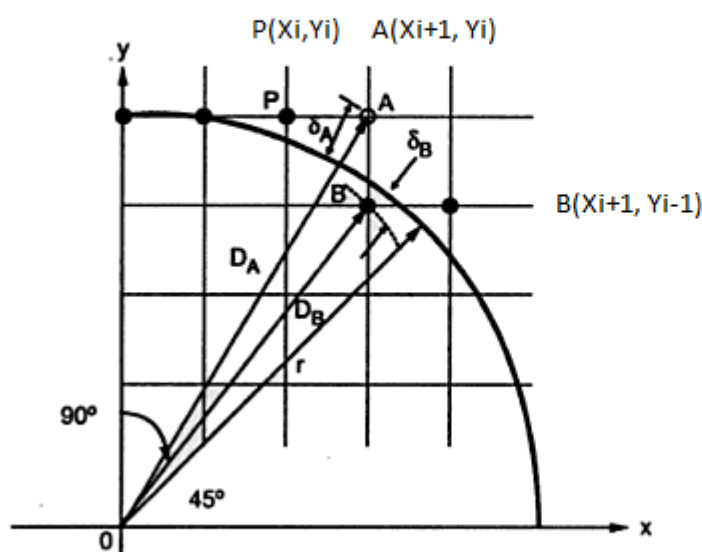
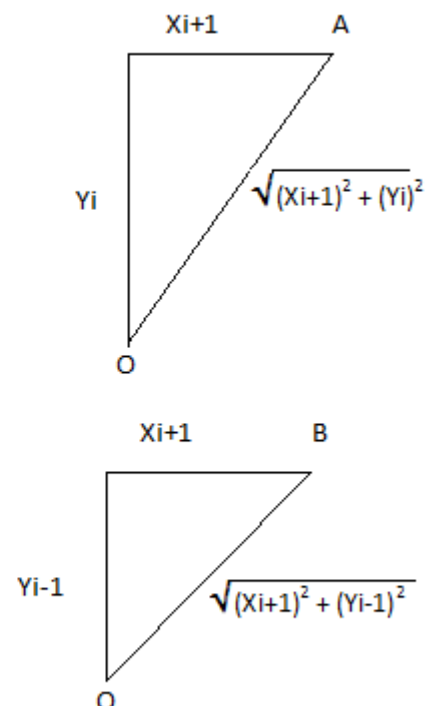


Fig. 2.25 Scan conversion with Bresenham's algorithm



Let us assume point P in Fig. 2.25 as a last scan converted pixel. Now we have two options either to choose pixel A or pixel B . The closer pixel amongst these two can be determined as follows.

The distances of pixels A and B from the origin are given as,

$$D_A = \sqrt{(x_{i+1})^2 + (y_i)^2} \quad \text{and}$$

$$D_B = \sqrt{(x_{i+1})^2 + (y_i - 1)^2}$$

Now, the distances of pixels A and B from the true circle are given as,

$$\delta_A = D_A - r \quad \text{and} \quad \delta_B = D_B - r$$

However, to avoid square root term in derivation of decision variable, i.e. to simplify the computation and to make algorithm more efficient the δ_A and δ_B are defined as,

$$\delta_A = D_A^2 - r^2 \quad \text{and}$$

$$\delta_B = D_B^2 - r^2$$

From Fig. 2.25, we can observe that δ_A is always positive and δ_B always negative. Therefore, we can define decision variable d_i as

$$d_i = \delta_A + \delta_B$$

and we can say that, if $d_i < 0$, i.e., $\delta_A < \delta_B$ then only x is incremented; otherwise x is incremented in positive direction and y is incremented in negative direction. In other words we can write,

$$\text{For } d_i < 0, \quad x_{i+1} = x_i + 1 \quad \text{and}$$

$$\text{For } d_i \geq 0, \quad x_{i+1} = x_i + 1 \quad \text{and} \quad y_{i+1} = y_i - 1$$

The equation for d_i at starting point, i.e. at $x = 0$ and $y = r$ can be simplified as follows,

$$\begin{aligned} d_i &= \delta_A + \delta_B \\ &= (x_i + 1)^2 + (y_i)^2 - r^2 + (x_i + 1)^2 + (y_i - 1)^2 - r^2 \\ &= (0 + 1)^2 + (r)^2 - r^2 + (0 + 1)^2 + (r - 1)^2 - r^2 \\ &= 1 + r^2 - r^2 + 1 + r^2 - 2r + 1 - r^2 \\ &= 3 - 2r \end{aligned}$$

Similarly, the equations for d_{i+1} for both the cases are given as,

$$\text{For } d_i < 0, \quad d_{i+1} = d_i + 4x_i + 6 \quad \text{and}$$

$$\text{For } d_i \leq 0, \quad d_{i+1} = d_i + 4(x_i - y_i) + 10$$

Algorithm to plot 1/8 of the circle

1. Read the radius (r) of the circle.
2. $d = 3 - 2r$
[Initialize the decision variable]
3. $x = 0, y = r$
[Initialize starting point]
4. do
{
 plot (x, y)
 if ($d < 0$) then
 {
 $d = d + 4x + 6$
 }
 else
 { $d = d + 4(x - y) + 10$
 $y = y - 1$
 }
 $x = x + 1$
} while ($x < y$)
5. Stop

The remaining part of circle can be drawn by reflecting point about y axis, x axis and about origin as shown in Fig. 2.18.

Therefore, by adding seven more plot commands after the plot command in the step 4 of the algorithm, the circle can be plotted. The remaining seven plot commands are :

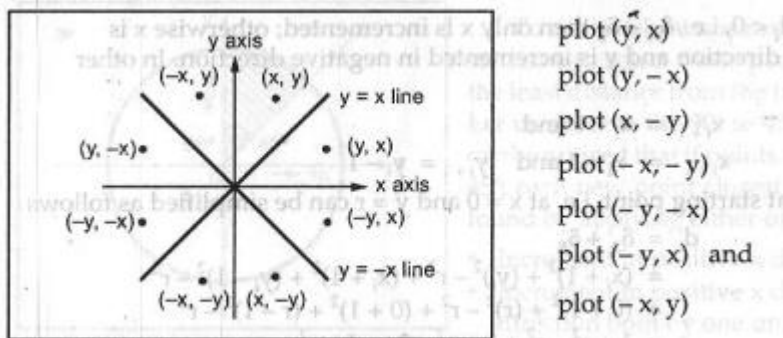


Fig. 2.18 Eight-way symmetry of the circle

2.6.3 Midpoint Circle Drawing Algorithm

The midpoint circle drawing algorithm also uses the eight-way symmetry of the circle to generate it. It plots 1/8 part of the circle, i.e. from 90° to 45° , as shown in the Fig. 2.27. As circle is drawn from 90 to 45° , the x moves in the positive direction and y moves in the negative direction. To draw a 1/8 part of a circle we take unit steps in the positive x direction and make use of decision parameter to determine which of the two possible y positions is closer to the circle path at each step. The Fig. 2.27 shows the two possible y positions (y_i and $y_i + 1$) at sampling position $x_i + 1$. Therefore, we have to determine whether the pixel at position $(x_i + 1, y_i)$ or at position $(x_i + 1, y_i - 1)$ is closer to the circle. For this purpose decision parameter is used. It uses the circle function ($f_{\text{circle}}(x, y) = x^2 + y^2 - r^2$) evaluated at the midpoint between these two pixels.

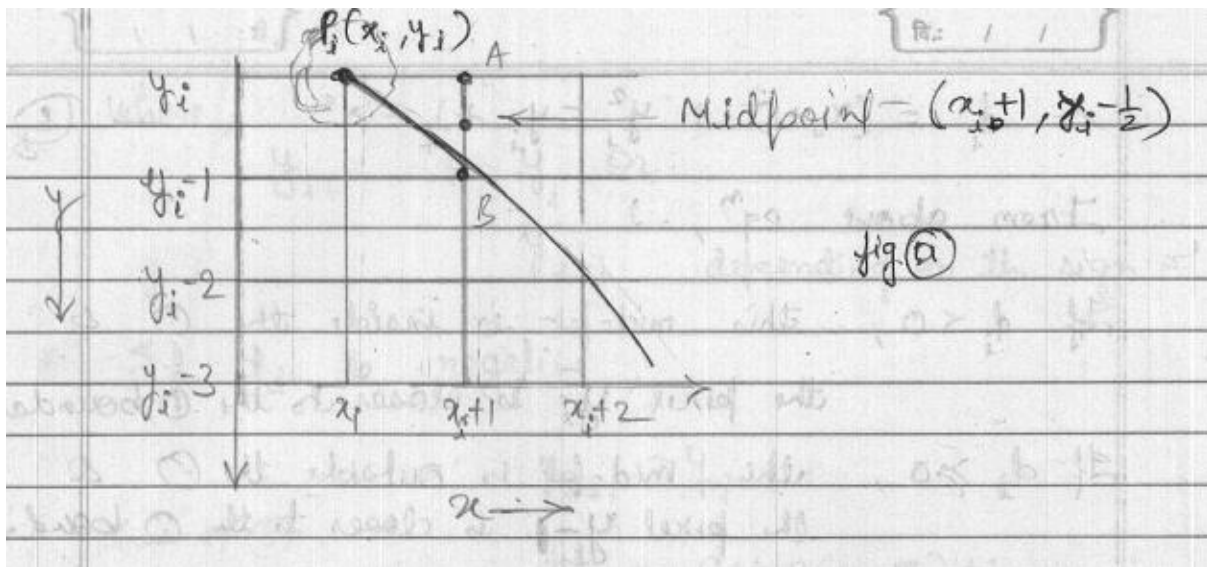


Fig. 2.27 Decision parameter to select correct pixel in circle generation algorithm

From above fig (a) point (x_i, y_i) is plotted (known pt. is plotted)

Now next, algorithm determines whether the pixel at position $(x_i + 1, y_i) = A$ or pixel at position $(x_i + 1, y_i - 1) = B$ is near to circle boundary and plot pixel whichever is near.

For this, decision parameter is used evaluated at the mid-point of these two pixels. From fig (a), co-ordinates of this mid-pt. are $(x_i + 1, y_i - \frac{1}{2})$

It uses the circle f^n .

$$f_{\text{circle}}(x, y) = x^2 + y^2 - r^2$$

putting values of mid-pt co-ordinates.

$$d_i = f_{\text{circle}}(x, y)$$

$$d_i = (x_i + 1)^2 + (y_i - \frac{1}{2})^2 - r^2 \quad \text{--- eqn (3a)}$$

$$d_i = f_{\text{circle}}(x_i + 1, y_i - \frac{1}{2})$$

$$= (x_i + 1)^2 + \left(y_i - \frac{1}{2}\right)^2 - r^2$$

$$= (x_i + 1)^2 + y_i^2 - y_i + \frac{1}{4} - r^2 \quad \dots (2.30)$$

If $d_i < 0$, this midpoint is inside the circle and the pixel on the scan line y_i is closer to the circle boundary. If $d_i \geq 0$, the midposition is outside or on the circle boundary, and $y_i - 1$ is closer to the circle boundary. The incremental calculation can be determined to obtain the successive decision parameters. We can obtain a recursive expression for the next decision parameter by evaluating the circle function at sampling position $x_{i+1} + 1 = x_i + 2$

$$\begin{aligned}
d_{i+1} &= f_{\text{circle}}(x_{i+1} + 1, y_{i+1} - \frac{1}{2}) \\
&= [(x_i + 1) + 1]^2 + \left(y_{i+1} - \frac{1}{2}\right)^2 - r^2 \\
&= (x_i + 1)^2 + 2(x_i + 1) + 1 + y_{i+1}^2 - (y_{i+1}) + \frac{1}{4} - r^2 \quad \dots (2.31)
\end{aligned}$$

Looking at equations (2.30) and (2.31) we can write

$$d_{i+1} = d_i + 2(x_i + 1) + (y_{i+1}^2 - y_i^2) - (y_{i+1} - y_i) + 1$$

where y_{i+1} is either y_i or y_{i-1} , depending on the sign of d_i .

If d_i is negative, $y_{i+1} = y_i$

$$\begin{aligned}
\therefore d_{i+1} &= d_i + 2(x_i + 1) + 1 \\
&= d_i + 2x_{i+1} + 1 \quad \dots (2.32)
\end{aligned}$$

If d_i is positive, $y_{i+1} = y_{i-1}$

$$\therefore d_{i+1} = d_i + 2(x_i + 1) + 1 - 2y_{i+1} \quad \dots (2.33)$$

The terms $2x_{i+1}$ and $-2y_{i+1}$ in equations (2.8) and (2.9) can be incrementally calculated as

$$\begin{aligned}
2x_{i+1} &= 2x_i + 2 \\
2y_{i+1} &= 2y_i - 2
\end{aligned}$$

The initial value of decision parameter can be obtained by evaluating circle function at the start position $(x_0, y_0) = (0, r)$.

$$\begin{aligned}
d_0 &= f_{\text{circle}}\left((0+1)^2 + \left(r - \frac{1}{2}\right)^2 - r^2\right) \\
&= 1 + \left(r - \frac{1}{2}\right)^2 - r^2 \\
&= 1.25 - r
\end{aligned}$$

Algorithm

1. Read the radius (r) of the circle
2. Initialize starting position as
$$x = 0$$
$$y = r$$
3. Calculate initial value of decision parameter as
$$d = 1.25 - r$$
4. do
 - { plot (x, y)
 -
 - if ($d < 0$)
 - { $x = x + 1$
 - $y = y$
 - $d = d + 2x + 1$
 - }
 - else
 - { $x = x + 1$
 - $y = y - 1$
 - $d = d + 2x + 2y + 1$
 - }
 -
 - while ($x < y$)
5. Determine symmetry points
6. Stop.

Polygons

A polyline is a chain of connected line segments. It is specified by giving the vertices (nodes) $P_0, P_1, P_2 \dots$ and so on. The first vertex is called the initial or starting point and the last vertex is called the final or terminal point, as shown in the Fig. 3.1 (a). When starting point and terminal point of any polyline is same, i.e. when polyline is closed then it is called polygon. This is illustrated in Fig. 3.1 (b).

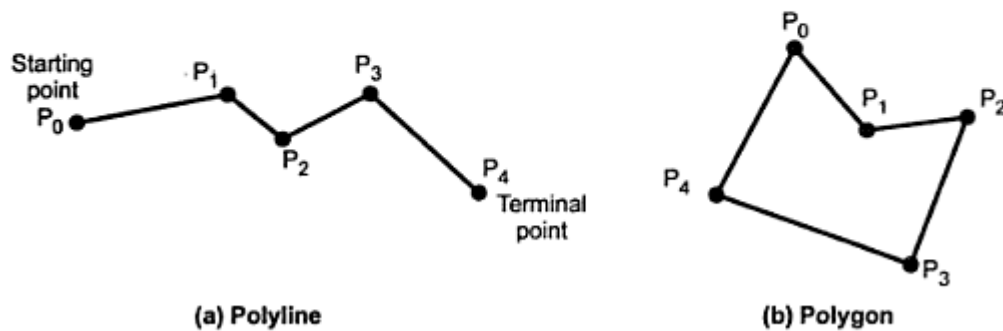


Fig. 3.1

3.2 Types of Polygons

The classification of polygons is based on where the line segment joining any two points within the polygon is going to lie. There are two types of polygons :

- Convex and
- Concave

A convex polygon is a polygon in which the line segment joining any two points within the polygon lies completely inside the polygon. The Fig. 3.2 shows the examples of convex polygons.

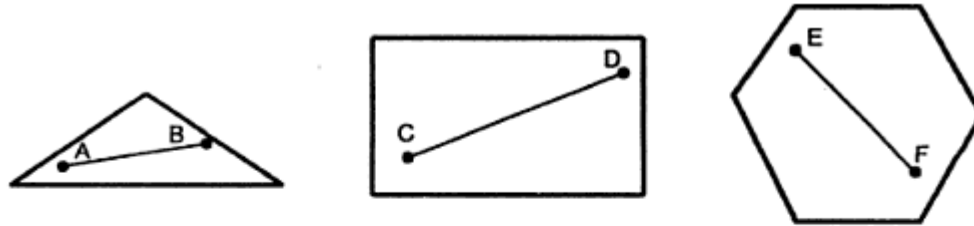


Fig. 3.2 Convex polygons

A concave polygon is a polygon in which the line segment joining any two points within the polygon may not lie completely inside the polygon. The Fig. 3.3 shows the examples of concave polygons.

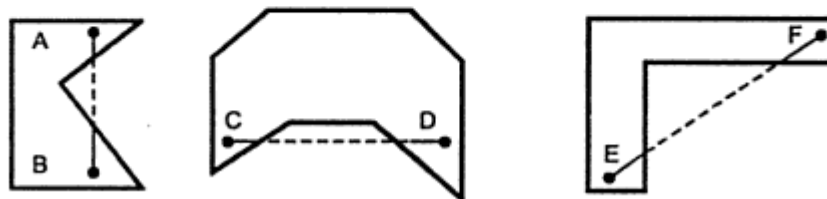


Fig. 3.3 Concave polygons

3.3 Representation of Polygons

We have seen that closed polyline is a polygon. Each polygon has sides and edges. The end points of the sides are called the polygon vertices. To add polygon to our **graphics** system, we must first decide how to represent it. There are three approaches to represent polygons according to the **graphics** systems :

- Polygon drawing primitive approach
- Trapezoid primitive approach
- Line and point approach

Some **graphics** devices supports polygon drawing primitive approach. They can directly draw the polygon shapes. On such devices polygons are saved as a unit. Some **graphics** devices support trapezoid primitive. In such devices, trapezoids are formed from two scan lines and two line segments as shown in the Fig. 3.4. Here, trapezoids are drawn **by** stepping down the line segments with two vector generators and, for each scan line, filling in all the pixels between them. Therefore every polygon is broken up into trapezoids and it is represented as a series of trapezoids.

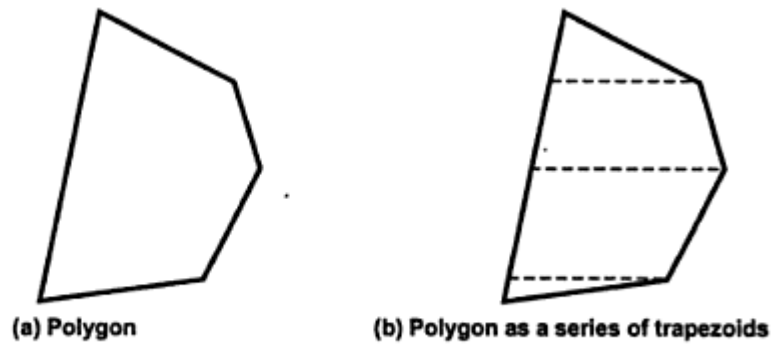
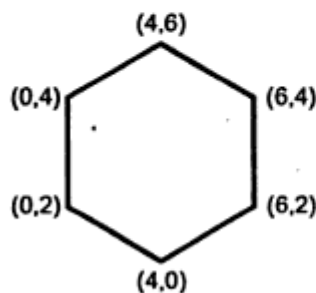


Fig. 3.4 Representation of polygon

Most of the other **graphics** devices do not provide any polygon support at all. In such cases polygons are represented using lines and points. A polygon is represented as a unit and it is stored in the display file. In a display file polygon can not be stored only with series of line commands because they do not specify how many of the following line commands are the part of the polygon. Therefore, new command is used in the display file to represent polygons. The opcode for new command itself specify the number of line segments in the polygon. The Fig. 3.5 shows the polygon and its representation using display file.



DF_OP	DF_x	DF_y
6	0	2
2	0	4
2	4	6
2	6	4
2	6	2
2	4	0
2	0	2

Fig. 3.5 Polygon and its representation using display file

3.4 Entering Polygons

Let us see how to enter polygon command and data into the display file. We know that, we have to enter number of sides and the coordinates of the vertex points. This information can be entered using a following algorithm.

Algorithm : Entering the polygon into the display file

1. Read AX and AY of length N
[AX and AY are arrays containing the vertices of the polygon and N is the number of polygon sides]
2. $i = 0$ [Initialize counter to count number of sides]
DF_OP [i] $\leftarrow$ N
DF_x [i] $\leftarrow$ AX [i]
DF_y [i] $\leftarrow$ AY [i]
 $i \leftarrow i + 1$
[Load polygon command]
3. do
{
DF_OP [i] $\leftarrow$ 2
DF_x [i] $\leftarrow$ AX [i]

DF_y [i] $\leftarrow$ AY [i]
 $i \leftarrow i + 1$
}
While ($i < N$) [Enter line commands]
4. DF_OP [i] $\leftarrow$ 2
DF_x [i] $\leftarrow$ AX [0]
DF_y [i] $\leftarrow$ AY [0]
[Enter last line command]
5. Stop

3.5 An Inside Test

Once the polygon is entered in the display file, we can draw the outline of the polygon. To show polygon as a solid object we have to set the pixels inside the polygon as well as pixels on the boundary of it. Now the question is how to determine whether or not a point is inside of a polygon. One simple method of doing this is to construct a line segment between the point in question and a point known to be outside the polygon, as shown in the Fig. 3.6. Now count how many intersections of the line segment with the polygon boundary occur. If there are an odd number of intersections, then the point in question is inside; otherwise it is outside. This method is called the **even-odd method** of determining polygon inside points.

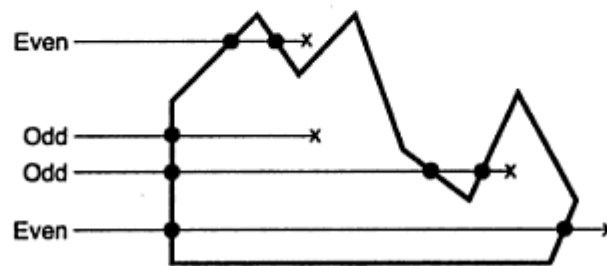


Fig. 3.6

If the intersection point is vertex of the polygon then we have to look at the other endpoints of the two segments which meet at this vertex. If these points lie on the same side of the constructed line, then the point in question counts as an even number of intersections. If they lie on opposite sides of the constructed line, then the point is counted as a single intersection. This is illustrated in Fig. 3.7.

Another approach to do the inside test is the **winding-number method**. Let us consider a point in question and a point on the polygon boundary. Conceptually, we can stretch a piece of elastic between these points and slide the end point on the polygon boundary for one complete rotation. We can then examine the point in question to see

how many times the elastic has wound around it. If it is wound at least once, then the point is inside. If there is no net winding, then the point is outside.

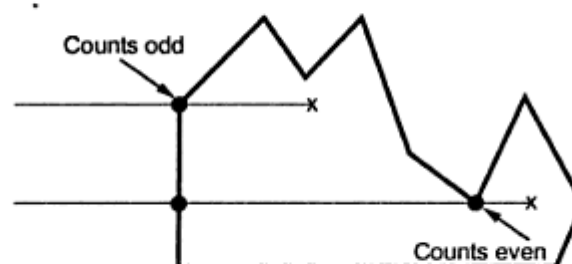


Fig. 3.7

Like even-odd method, in winding number method we have to picturise a line segment running from outside the polygon to the point in question and consider the polygon sides which it crosses. Here, instead of just counting the intersections, we have to give a **direction number** to each boundary line crossed, and we have to sum these direction numbers. The direction number indicates the direction the polygon edge was drawn relative to the line segment we have constructed for the test. This is illustrated in Fig. 3.8

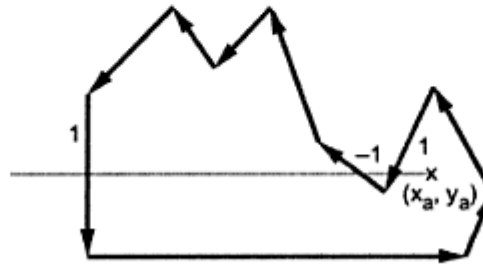


Fig. 3.8

As shown in the Fig. 3.8, point (x_a, y_a) is a test point and line $y = y_a$ is a horizontal line runs from outside the polygon to point (x_a, y_a) . The polygon edges crossed by this line could be drawn in two ways. The edge could be drawn starting below the line, cross it, and end above the line or starting above the line, cross it, and end below the line. In first case we have to give direction number as -1 and in the second case we have give direction number as 1 . After giving the direction numbers we have to take sum of these direction numbers which indicates whether the point in inside the polygon or not. The sum of the direction numbers for the sides that cross the constructed horizontal line segment is called the winding number for the point in question. For polygons or two dimensional objects, the point is said to be inside when the value of winding number is nonzero.

3.6 Polygon Filling

Filling the polygon means highlighting all the pixels which lie inside the polygon with any colour other than background colour. Polygons are easier to fill since they have linear boundaries.

There are two basic approaches used to fill the polygon. One way to fill a polygon is to start from a given "seed", point known to be inside the polygon and highlight outward from this point i.e. neighbouring pixels until we encounter the boundary pixels. This approach is called **seed fill** because colour flows from the seed pixel until reaching the polygon boundary, like water flooding on the surface of the container. Another approach to fill the polygon is to apply the inside test i.e. to check whether the pixel is inside the polygon or outside the polygon and then highlight pixels which lie inside the polygon. This approach is known as **scan-line algorithm**. It avoids the need for a seed pixel but it requires some computation. Let us see these two methods in detail.

3.6.1.2 Flood Fill Algorithm

Sometimes it is required to fill in an area that is not defined within a single colour boundary. In such cases we can fill areas **by** replacing a specified interior colour instead of searching for a boundary colour. This approach is called a flood-fill algorithm. Like boundary fill algorithm, here we start with some seed and examine the neighbouring pixels. However, here pixels are checked for a specified interior colour instead of boundary

colour and they are replaced **by** new colour. Using either a 4-connected or 8-connected approach, we can step through pixel positions until all interior point have been filled. The following procedure illustrates the recursive method for filling 8-connected region using flood-fill algorithm.



Fig. 3.9

Procedure : flood_fill (x, y, old_colour, new_colour).

```
{
    if ( getpixel (x, y) = old_colour)
    {
        putpixel (x, y, new_colour);
        flood_fill (x + 1, y, old_colour, new_colour);
        flood_fill (x - 1, y, old_colour, new_colour);
        flood_fill (x, y + 1, old_colour, new_colour);
        flood_fill (x, y - 1, old_colour, new_colour);
        flood_fill (x + 1, y + 1, old_colour, new_colour);
        flood_fill (x - 1, y - 1, old_colour, new_colour);
        flood_fill (x + 1, y - 1, old_colour, new_colour);
        flood_fill (x - 1, y + 1, old_colour, new_colour);
    }
}
```

Note : 'getpixel' function gives the colour of specified pixel and 'putpixel' function draws the pixel with specified colour.

Comparison Chart

BASIS FOR COMPARISON	FLOOD-FILL ALGORITHM	BOUNDARY-FILL ALGORITHM
Basic	It can have an area containing several colours.	It defines the area with a single colour.
Painting process	A random colour can be used to colour the interior portion then the old one is replaced with a new one.	Interior points are coloured by continuously searching for the boundary colour.
Memory consumption	High	Low
Speed	Comparatively slower	Fast
Algorithm complexity	Simple relatively	Complicated

3.6.2 Scan Line Algorithm

Recursive algorithm for seed fill methods have got two difficulties : The first difficulty is that if some inside pixels are already displayed in fill colour then recursive branch terminates, leaving further internal pixels unfilled. To avoid this difficulty, we have to first change the colour of any internal pixels that are initially set to the fill colour before applying the seed fill procedures. Another difficulty with recursive seed fill methods is that it cannot be used for large polygons. This is because recursive seed fill procedures require stacking of neighbouring points and in case of large polygons stack space may be insufficient for stacking of neighbouring points. To avoid this problem more efficient method can be used. Such method fills horizontal pixel spans across scan lines, instead of proceeding to 4-connected or 8-connected neighbouring points. This is achieved by identifying the rightmost and leftmost pixels of the seed pixel and then drawing a horizontal line between these two boundary pixels. This procedure is repeated with changing the seed pixel above and below the line just drawn until complete polygon is filled. With this efficient method we have to stack only a beginning position for each horizontal pixel span, instead of stacking all unprocessed neighbouring positions around the current position.

The Fig. 3.11 illustrates the scan line algorithm for filling of polygon. For each scan line crossing a polygon, this algorithm locates the intersection points of the scan line with the polygon edges. These intersection points are then sorted from left to right and the corresponding positions between each intersection pair are set to the specified fill colour.

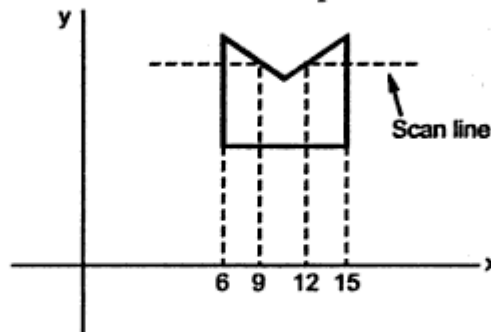


Fig. 3.11

In Fig. 3.11, we can see that there are two stretches of interior pixels from $x = 6$ to $x = 9$ and $x = 12$ to $x = 15$. The scan line algorithm first finds the largest and smallest y values of the polygon. It then starts with the largest y value and works its way down, scanning from left to right, in the manner of a raster display.

The important task in the scan line algorithm is to find the intersection points of the scan line with the polygon boundary. When intersection points are even, they are sorted from left to right, paired and pixels between paired points are set to the fill colour. But in some cases intersection point is a vertex. When scan line intersects polygon vertex a special handling is required to find the exact intersection points. To handle such cases, we must look at the other endpoints of the two line segments of the polygon which meet at this vertex. If these points lie on the same (up or down) side of the scan line, then the point in question counts as an even number of intersections. If they lie on opposite sides of the scan line, then the point is counted as single intersection. This is illustrated in Fig.3.12.

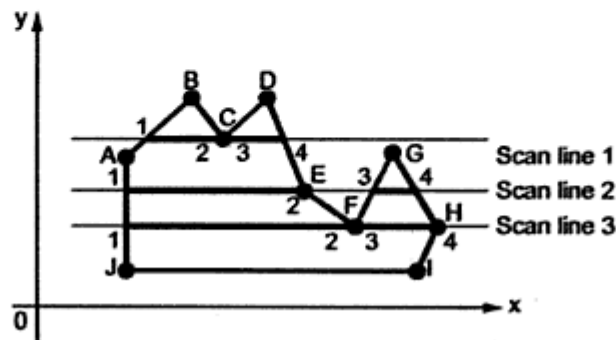


Fig. 3.12 Intersection points along the scan line that intersect polygon vertices

As shown in the Fig. 3.12, each scan line intersects the vertex or vertices of the polygon. For scan line 1, the other end points (B and D) of the two line segments of the polygon lie on the same side of the scan line, hence there are two intersections resulting two pairs : 1 - 2 and 3 - 4. Intersections points 2 and 3 are actually same points. For scan

For scan line 2 the other endpoints (D and F) of the two line segments of the polygon...

(D and F) of the two line segments of the polygon lie on the opposite sides of the scan line, hence there is a single intersection resulting two pairs : 1 - 2 and 3 - 4. For scan line 3, two vertices are the intersection points. For vertex F the other end points E and G of the two line segments of the polygon lie on the same side of the scan line whereas for vertex H, the other endpoints G and I of the two line segments of the polygon lie on the opposite side of the scan line. Therefore, at vertex F there are two intersections and at vertex H there is only one

We have seen that it is necessary to calculate x intersection points for scan line with every polygon side. We can simplify these calculations by using **coherence properties**. A coherence property of a scene is a property of a scene by which we can relate one part of a scene with the other parts of a scene. Here, we can use a slope of an edge as a coherence property. By using this property we can determine the x intersection value on the lower scan line if the x intersection value for current scan line is known. This is given as

$$x_{i+1} = x_i - \frac{1}{m}$$

where m is the slope of the edge

As we scan from top to bottom value of y coordinates between the two scan line changes by 1.

$$y_{i+1} = y_i - 1$$

Many times it is not necessary to compute the x intersections for scan line with every polygon side. We need to consider only the polygon sides with endpoints straddling the current scan line. See Fig. 3.13.

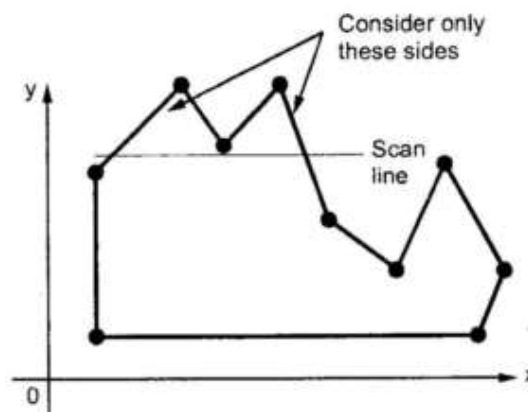


Fig. 3.13 Consider only the sides which intersect the scan line

It will be easier to identify which polygon sides should be tested for x-intersection, if we first sort the sides in order of their maximum y value. Once the sides are sorted we can process the scan lines from the top of the polygon to its bottom producing an active edge list for each scan line crossing the polygon boundaries. The active edge list for a scan line

line contains all edges crossed **by** that scan line. The Fig. 3.14 shows sorted edges of the polygon with active edges.

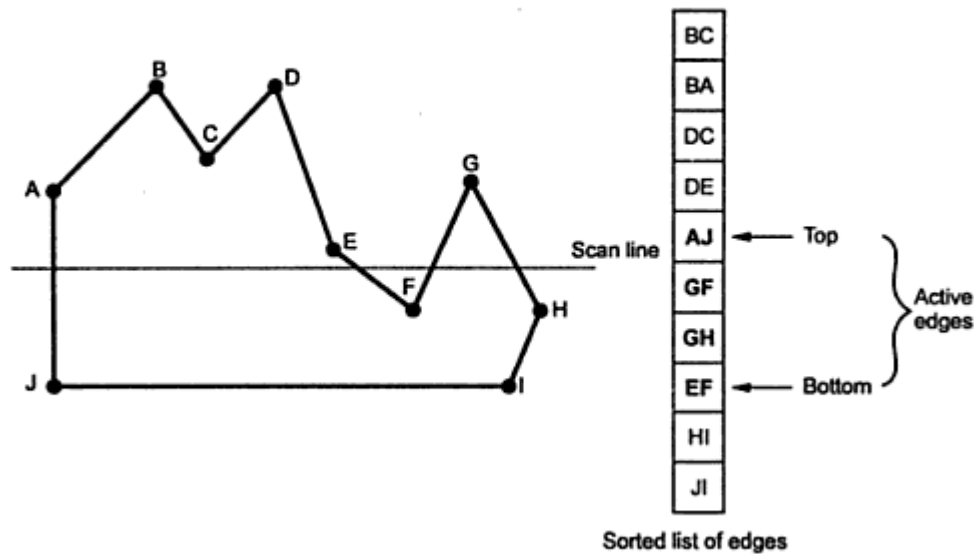


Fig. 3.14 Sorted edges of the polygon with active edges

In summary, a scan line algorithm for filling a polygon begins **by** ordering the polygon sides on the largest y value. It begins with the largest y value and scans down the polygon. For each y, it determines which sides can be intersected and finds the x values of these intersection points. It then sorts, pairs and passes these x values to a line drawing routine.

Scan Line Conversion Algorithm for Polygon Filling :

1. Read n, the number of vertices of polygon
2. Read x and y coordinates of all vertices in array x[n] and y[n].
3. Find y_{min} and y_{max} .
4. Store the initial x value (x_1) y values y_1 and y_2 for two endpoints and x increment Δx from scan line to scan line for each edge in the array edges [n] [4].

While doing this check that $y_1 > y_2$, if not interchange y_1 and y_2 and corresponding x_1 and x_2 so that for each edge, y_1 represents its maximum y coordinate and y_2 represents its minimum y coordinate.

5. Sort the rows of array, edges [n] [4] in descending order of y_1 , descending order of y_2 and ascending order of x_2 .
6. Set $y = y_{max}$
7. Find the active edges and update active edge list :

if ($y > y_2$ and $y \leq y_1$)
 { edge is active }

else

{ edge is not active }

8. Compute the x intersects for all active edges for current y value [initially x-intersect is x_1 and x intersects for successive y values can be given as

$$x_{i+1} \leftarrow x_i + \Delta x$$

$$\text{where } \Delta x = -\frac{1}{m} \text{ and } m = \frac{y_2 - y_1}{x_2 - x_1} \text{ i.e. slope of a line segment}$$

9. If x intersect is vertex i.e. x-intersect = x_1 and $y = y_1$ then apply vertex test to check whether to consider one intersect or two intersects. Store all x intersects in the x-intersect [] array.
10. Sort x-intersect [] array in the ascending order,
11. Extract pairs of intersects from the sorted x-intersect [] array.
12. Pass pairs of x values to line drawing routine to draw corresponding line segments
13. Set $y = y - 1$
14. Repeat steps 7 through 13 until $y \geq y_{\min}$.
15. Stop

In step 7, we have checked for $y \leq y_1$ and not simply $y < y_1$. Hence step 9 a becomes redundant. Following program takes care of that.